

Capabilities to Catch 'em All: Unifying Multitier, Choreographic, and Aggregate Programming

ANONYMOUS AUTHOR(S)

Multiparty languages adopt a global perspective on programming distributed systems, abstracting over individual components and interaction. For example, choreographic programming ensures protocol correctness, multitier programming allows defining functionalities across network boundaries, and aggregate computing enables modeling collective behavior over dynamic ensembles of devices. Such paradigms offer distinct abstractions and safety guarantees and are typically used in isolation. This separation makes it difficult to take advantage of such paradigms within a single system while preserving their respective correctness properties.

In this paper, we show that capabilities can be used as a unifying abstraction for multiparty programming. We demonstrate that multitier, choreographic, and aggregate programming can each be modeled as capabilities that regulate access to paradigm-specific operations, and that these capabilities can be safely composed to express systems that combine multiple paradigms.

We present Party, a programming framework based on such a design that integrates multitier, choreographic, and aggregate programming. Our approach enforces correct placement and communication at compile time and prevents common classes of distributed programming errors statically. We formalize the core of Party in a typed calculus, and prove type soundness. Our evaluation on 46 use cases from literature and two larger case studies – one implemented from scratch and one ported from a GitHub third-party project – shows that Party's capabilities-based design can capture the abstractions of multitier, choreographic, and aggregate programming, their safety features, as well as their safe composition in distributed applications.

CCS Concepts: • **Software and its engineering** → **Distributed programming languages**; *Formal language definitions*; *Language features*.

Additional Key Words and Phrases: Distributed programming, choreographic programming, multitier programming, aggregate computing

1 Introduction

Over the last decade, researchers have increasingly explored multiparty programming paradigms that adopt a holistic view of distributed systems [7, 51, 67]. Rather than programming each participant in isolation and reasoning locally from the perspective of a single node, these paradigms encourage developers to describe system behavior globally, at the level of the entire distributed computation. By supporting interactions, placement, and collective behavior at the language level, such approaches aim to reduce incidental complexity and make cross-host coordination structure explicit in the program. This global perspective contrasts with traditional distributed programming models, where developers must manually orchestrate communication, synchronization, and data distribution from the viewpoint of individual peers.

We focus on three main flavors of multiparty programming that have gained interest recently, each emphasizing a different dimension of global reasoning: choreographic programming, multitier programming, and aggregate computing.

Choreographic programming [51] focuses on the global specification of interactions among distributed participants. *Multitier programming* [67] centers on the explicit placement of computations across architectural tiers (e.g., client, server, edge). *Aggregate computing* [7] models large ensembles of devices as a single computational entity, defining high-level collective behaviors that emerge from local device interactions. We introduce each paradigm in detail in [Section 2](#).

Despite these paradigms sharing the common foundation of providing a holistic view on the distributed system and reifying distributed components/parties into the language, the literature has largely treated these paradigms in isolation [2, 19, 23, 42, 50, 59, 60, 63, 64] and a principled unifying framework that integrates them is still missing. In practice, however, realistic distributed systems

rarely adhere to a single programming model. A system may require structured protocol interactions between well-defined roles, explicit control over architectural placement, and collective coordination across dynamic ensembles of devices – all within the same application. Today, developers must either commit to one paradigm and encode the others indirectly, or manually combine heterogeneous frameworks with limited guarantees about their interaction.

This fragmentation raises both conceptual and technical challenges. Each paradigm provides its own abstractions, assumptions about connectivity, and static guarantees. Without a common foundation, developers lack a unified way to reason about how these abstractions interact, and type systems cannot enforce correctness properties across paradigm boundaries.

Contributions. This paper introduces Party, a unifying framework that brings together choreographic, multitier and aggregate programming by adopting *capabilities* as a common abstraction. Recently, capability and effect systems have gained significant attention in the programming languages community [10, 11, 20, 25] as they enable precise reasoning about the effects a function may perform and ensure that effects are only executed when the required capabilities are available. Building on this idea, we propose to model the three paradigms as capabilities, and show how they can be integrated within a single framework using the concept of *placement types* [65], which allow statically specifying where values and computations reside within the system architecture, enabling type-safe reasoning about distribution and placement. The main contributions of this paper are:

- We propose capabilities as a foundational abstraction to unify multiparty languages, namely choreographic, multitier, and aggregate, and support their composition and safety features.
- We present the design of Party, a unifying framework based on the idea above that integrates choreographic languages, multitier programming, and aggregate computing and enables their combined use within the same system.
- We provide a formalization of the core concepts of Party and prove that the interaction of remotely and locally placed values is sound across the different paradigms.
- We implement Party in Scala, showcasing its practical applicability in real-world scenarios.
- We evaluate our approach on multiple case studies from literature and open-source systems to assess its expressiveness, safety, inter-paradigm integration, portability across languages and capability systems.

2 Background and Running Example

This section introduces the three multiparty programming paradigms – multitier, choreographic, and aggregate computing – through a running example. For each, we first introduce the paradigm, including its core abstractions and then show how the paradigm applies to the running scenario.

2.1 Scenario: CityEvt System

In the CityEvt scenario, participants join the event through a smartphone app. During the event, they receive recommendations about nearby points of interest, e.g., restaurants and museums. For safety, the system monitors the crowding level of the event, reporting any critical situation to the organizers to take action. To preserve the participants' privacy, the system does not collect individual positions. Instead, a decentralized, collective algorithm, based on proximity perception, estimates the crowding level of the event.

The system employs a **cloud** service for storing the participants' data, managing their registrations, and collecting analytics about the event. **Smartphones** are used by the participants to register for the event, receive recommendations, and report the crowding level,

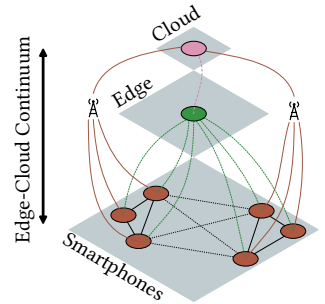


Fig. 1. CityEvt architecture.

while **edge servers** are used to provide ad hoc recommendations to the participants. Figure 1 shows the architecture of CityEvt. We identify three parts of the system: (i) user registration and rejoining, (ii) crowding level monitoring, and (iii) recommendation system. Each part is best addressed by a different programming paradigm, as we discuss next. We introduce each paradigm with its role in the running example.

2.2 User Registration via Multitier Programming

Multitier Programming [67] adopts a global perspective centered on the explicit placement of computations across architectural tiers (e.g., client, server, edge). The system logic is written as a unified program and annotations on the program structure (i.e., on nested lexical scopes) indicate where expressions execute. Recent multitier languages adopt an explicit architectural model and allow control flow and data to move across tiers.

In the ScalaLoci multitier language [65], *peers* are the entities on which placement can occur and *ties* between them define possible communication among peers. Each tie features a cardinality constraint: a peer is tied to a *single* or *multiple* other peers. The following code shows the complete architecture specification that reflects the running CityEvt example depicted in Figure 1 in ScalaLoci:

```

1 @peer type Cloud <: { type Tie <: Multiple[Phone] & Single[Edge] }
2 @peer type Edge <: { type Tie <: Multiple[Phone] & Single[Cloud] }
3 @peer type Phone <: { type Tie <: Multiple[Phone] & Single[Cloud] & Multiple[Edge] }

```

ScalaLoci models peers as structural types with a **Tie** type member, as **type Peer <: { type Tie }**. By using intersection types (**&**), we can refine the **Tie** specification to define complex architectures. Line 1 defines the **Cloud** peer tied to multiple **Phone** peers and a single **Edge** peer. Line 2 defines the **Edge** tied to multiple **Phone** peers and a single **Cloud**. Line 3 defines the **Phone** peer tied to a single **Cloud**, multiple **Edge** peers and multiple other **Phone** peers, allowing phone-to-phone communication. While this scenario ties peers to all other peer types, any attempt to access another **Cloud** peer from the **Cloud** would be rejected by the type system, as there is no tie between them.

User Registration. In CityEvt, multitier programming is a good fit for the registration logic which spans smartphones and cloud peers. The following ScalaLoci code illustrates the registration logic:

```

1 @multitier object RegistrationSystem:
2   /* Architecture above */
3   val registeredIds: Map[Remote[Phone], Id] on Cloud = /* ... */
4
5   on[Cloud]:
6     remote[Phone].joined foreach: smartphone =>
7       val registrationRequest = on(smartphone) { /* fill form and submit */ }.asLocal
8       registeredIds += smartphone -> registerUser(registrationRequest)
9
10  def id: Option[Id] on Phone =
11    (on[Cloud] sbj { smartphone => registeredIds.get(smartphone) }).asLocal

```

After registration, the smartphone retrieves its ID from the cloud. Values are annotated with *placement types* of the form **T on P**, indicating that a value of type **T** resides on peer **P**. The **on** construct (Lines 5 and 11) introduces a scope placed on a peer. The tier-crossing operator **asLocal** transfers placed values across peers, while **asLocalAll** collects values from multiple remote peers. The type system ensures that cross-tier accesses respect the architectural constraints. The compiler automatically identifies the code for each peer, separates it and injects the networking code.

2.3 Crowding Level Monitoring via Aggregate Computing

Aggregate Computing [7] models large ensembles of devices as a single computational entity. Aggregate programs define high-level collective behaviors emerging from local interactions where collective operators enable developers to reason about system-wide properties such as self-organization, adaptation, and convergence. The execution model is from the *field calculus* [4] based on *rounds*: devices independently and periodically execute the same program, exchanging information with neighbors and updating their local state. This execution model naturally applies to scenarios with loosely coupled devices, like sensor networks, IoT systems, and swarm robotics [7, 31, 32].

There are three main constructs: **rep** defines stateful computations across device rounds, allowing each device to maintain a local state that evolves over time; **nbr** enables the exchange of information between neighboring devices, building the so-called *neighboring field*; **branch** partitions the device network, enabling different behaviors for different subsets of devices aligned to the same condition. Beyond these primitives, aggregate programs typically rely on higher-level building blocks such as G (gradient-based propagation), S (sparse leader election) or C (distributed collection) [7], which preserve self-stabilization properties [4]. The gradient operator G, for example, can be used to spread information along a gradient from a sink device by specifying a propagation function and a distance metric. G can be implemented by combining **rep** and **nbr**:

```
1 def G[V: Bounded](src: Boolean, field: V, acc: V => V, metric: => Double) =
2   val (_, result) = rep(Double.MaxValue, field): dv =>
3     mux(src)(0.0, field):
4       minHoodPlus:
5         val (d, v) = nbr(dv)
6         (d + metric, acc(v))
7   result
```

The `distanceTo` operator – used in the next example – is a specialization of G that computes the distance to a source device, using the metric as the distance between neighbors, and the sum of the previous distance and the closeness of the neighbor as the accumulation function.

Crowding Level Monitoring. The crowding level monitoring reports any critical situation to the organizers. This level is estimated by a collective algorithm, based on the mutual interaction of the participants’ smartphones [7]. Because of variable network connectivity and the need for a privacy-preserving, decentralized approach to crowding estimation, aggregate computing suits this part of the system naturally. We illustrate the aggregate computation for crowd density estimation:

```
1 def crowdingLevel(): Double =
2   val id = S(300)
3   val distanceToLeader: Double = distanceTo(id)
4   val people = // get number of people detected by local sensors
5   val count = C(distanceToLeader, people, _ + _)
6   val radius = C(distanceToLeader, distanceToLeader, _ max _)
7   count / (math.Pi * radius * radius)
```

Line 2 uses the S combinator to create a spatial region of 300 meters, electing a device ID as the region “leader”; Line 3 computes the distance to the leader using `distanceTo`; Lines 5 and 6 use the C combinator to aggregate values up the gradient towards the leader; Line 5 sums the number of people across the region; Line 6 computes the maximum distance; and Line 7 computes the density as count divided by area. This algorithm preserves self-stabilizing and self-organizing properties [62], adapting to changes (e.g., devices joining or leaving) and converging to a correct estimate of the crowding level without explicit coordination.

2.4 Recommendations System via Choreographic Programming

Choreographic Programming [51] specifies communication protocols with correctness guarantees, such as deadlock freedom and protocol compliance by construction, ensuring that the resulting protocol is free of implicit data dependencies and enabling safe endpoint projection [13]. In a choreographic program, values on one peer can only be extracted on the same peer. Transferring data between peers requires explicit inter-peer communication, expressed through explicit operators such as **comm** (point-to-point), **multicast** (one-to-many) or **broadcast** (one-to-all).

Recommendation System. The registered participants receive recommendations about nearby points of interest by interacting with the edge server and sharing their ID, which is used to authenticate the users via the cloud service. The edge server then provides recommendations via the smartphone application. As authentication and recommendation require a complex protocol that combines interactions between the cloud, edge and smartphone components, the choreographic programming model is ideal. It allows developers to express the complete protocol, making the communication explicit and providing strong correctness guarantees. We illustrate an implementation with choreographic programming in ChoRus [6], which is based on Rust:

```
1 type L = LocationSet!(Cloud, Edge, Phone);
2 fn run(self, op: &impl ChoreoOp<Self::L>) {
3   let id_at_smartphone = op.locally(Phone, |_| get_user_id());
4   let id_at_cloud = op.comm(Phone, Cloud, id_at_smartphone);
5   let auth_at_cloud = op.locally(Cloud, |un| check_auth(un.unwrap(&id_at_cloud)));
6   let auth_at_edge = op.comm(Cloud, Edge, auth_at_cloud);
7   let rec_at_edge = op.locally(Edge, |un| get_recommendations(un.unwrap(&auth_at_edge)));
8   let rec_at_smartphone = op.comm(Edge, Phone, rec_at_edge);
9   op.locally(Phone, |un| display(un.unwrap(&rec_at_smartphone)));
10 }
```

The choreography implements the recommendation protocol as a sequence of explicit communication and local computation steps. Lines 3, 5, 7 and 9 use **locally** to place computations on specific peers. Lines 4, 6 and 8 use **comm** to explicitly transfer values between peers.

2.5 The Challenge: Combining Multiparty Solutions

In the CityEvt scenario, different parts of a distributed application adopt different programming paradigms. User registration follows a client-server pattern suited to multitier programming; crowd-estimation relies on collective self-organization best expressed through aggregate computing; and the recommendation workflow involves structured multiparty communication captured by choreographic programming. Yet, these paradigms have traditionally been developed and used in isolation. No existing solution provides a principled way to combine them within a single system, offering their respective features and preserving their respective correctness guarantees. In practice, developers must either adopt a single paradigm and encode the others indirectly, or manually combine heterogeneous frameworks with limited guarantees about their interaction. In the next section, we show how capabilities provide a unifying abstraction that enables the safe composition of multitier, choreographic, and aggregate programming within a single framework.

3 Party: Multiparty Programming with Capabilities

We present Party, a programming framework where capabilities provide a unifying abstraction for multiparty programming. We model each paradigm as a capability, and placement types serve as the common foundation for their composition. With this approach, the developer is free to choose the most suitable paradigm for each part of the system.

A capability is a value representing the authority to perform a class of operations. Capability-based designs require such values to invoke privileged functionality, ensuring that authority flows

through explicit program structure rather than global state. While capabilities come in many forms – static tokens for memory safety [25], reference capabilities for data-race freedom [20], or abstractions for side-effect management [11] – our approach relies on object capabilities, i.e., ordinary program values tracked in the type system [10]. Our prototype implementation expresses capabilities via Scala’s *contextual parameters*: methods declare required capabilities with the `using` keyword and type system resolves them implicitly from the surrounding (lexical) context. Operations of a given multiparty paradigm require the corresponding capability to be in scope, separating scopes associated with different paradigms, preventing unintended interactions while enabling structured composition of different paradigms via placement types as designated integration interfaces.

3.1 Placement Capability

Placement types were introduced in multitier programming to specify the classes of peers and their interconnections (ties). In Party, we adopt the concept of placement types across all three paradigms to provide a common way to reason about where values and computations reside in the system. The `Placement` capability provides two operations: (i) the ability to place values on specific peers through the `on` operation and (ii) the ability to extract values placed on the local peer through the `take` operation. The following snippet shows an example of the placement capability in Party:

```
1 def helloOnSmartphone(using Placement): String on Phone = on[Phone] { "Hello" }
2 def reply(using Placement): String on Phone = on[Phone]:
3   val hello: String = take(helloOnSmartphone)
4   s"${hello}, Smartphone!"
```

In the `helloOnSmartphone` function, `on` places the value `"Hello"` on the `Phone` peer. Hence, the return type of `helloOnSmartphone` is `String on Phone`. The `reply` function then uses `take` to extract the value placed on the `Phone` peer. The functions require the `Placement` capability (Lines 1 and 2), which provides the operations to create and access placement types.

Extracting a placed value is statically verified by the type system, ensuring that the `take` operation is only performed on the peer where the value is placed (see Section 3.6). The value placement capability does not provide any interaction between peers; such interactions are handled by each paradigm using its own mechanisms, as we will describe in the next sections.

3.2 Multitier Capability

We now show multitier programming in Party through the `Multitier` capability, which enables the tier-crossing operations `asLocal` and `asLocalAll` on placed values as capability-guarded constructs. The former allows transferring a value from a single remote peer to the local peer; the latter allows aggregating values from multiple remote peers. In multitier programming, communication can appear anywhere inside placed scopes where access to a remote value is required. We now revisit *user registration* (Section 2.2), based on Party’s multitier capability:

```
1 def registration(userId: String)(using Placement, Multitier) =
2   val registrationRequest: Form on Phone = on[Phone] { /* fill form */ }
3   val ids: Map[Remote[Phone], UUID] on Cloud = on[Cloud]:
4     val request = asLocalAll(registrationRequest)
5     request.view.mapValues(registerUser).toMap
6   val id: UUID on Phone = on[Phone]:
7     val localId = getId(localAddress)
8     asLocal(ids).get(localId)
```

Line 1 requires the `Placement` and `Multitier` capabilities. Line 2 places the registration request on the `Phone`. In Line 4, the `Cloud` peer accesses the registration request from the `Phone` peers

through using **asLocalAll** and registers the users through `registerUser`. In [Line 8](#), the **Phone** peer retrieves the registration token corresponding to its user ID by using the **asLocal** operator.

3.3 The Aggregate Computing Capability

We now show how Party models aggregate computing with the **Collective** capability, which provides the field calculus constructs **rep**, **nbr**, and **branch** as capability-guarded operations. We revisit the *crowd level monitoring* ([Section 2.3](#)):

```
1 def crowding(using Placement, Collective): Signal[Double] on Phone = Collective[Phone](1.s):
2   val distanceToLeader = distanceTo(S(300)) // 300 meters wide
3   val people = // get number of people detected by local sensors
4   val count = C(distanceToLeader, people, _ + _) // count devices
5   val radius = C(distanceToLeader, distanceToLeader, _ max _) // max distance
6   count / (math.Pi * radius * radius) // density = count / area
```

To define the crowd density estimation, the `crowding` function declares the **Collective** capability. We define a **Collective** computation on the **Phone** peer type with a 1-second round duration ([Line 1](#)). Inside the collective block, the crowd density estimation is implemented using collective building blocks. The return value is of the type `Signal[Double] on Phone`, representing a stream of density values updated in every collective round placed on the **Phone** peer type.

3.4 Choreography Capability

We now show choreographic programming in Party. The **Choreography** capability provides the choreographic operations **comm**, **multicast** and **broadcast**. The **comm** operation involves a single sender and a single receiver, requiring both peers to have a single tie to each other, while the **multicast** operation involves one sender and multiple receivers, enabling one-to-many communication. The **broadcast** operation makes a value available to all peers. We revisit the *recommendation system* ([Section 2.4](#)), focusing on the edge server and cloud components of the system, which are responsible for processing user data and providing personalized recommendations:

```
1 def retrievePreference(using Placement, Choreography) = Choreography:
2   val smartphoneToken = on[Phone] { /* get id */ }
3   val cloudToken = comm[Phone, Cloud](smartphoneToken)
4   val isRegistered = on[Cloud] { take(cloudToken) /* check if registered */ }
5   val shouldRecommend = comm[Cloud, Edge](isRegistered)
6   val recommendation = on[Edge]:
7     if take(shouldRecommend) then { /* provide recommendation */ }
8   val recommOnSmartphone = comm[Edge, Phone](recommendation)
9   on[Phone] { take(recommOnSmartphone) /* use recommendation */ }
```

The `retrievePreference` function models the choreography for retrieving user preferences from the cloud. Initially, the user ID is on the **Phone**, and it is sent to the **Cloud** through **comm**. The **Cloud** extracts the placed value through **take** to obtain the user preferences. The result is sent back to the **Edge** peer through another **comm**. Finally, the **Edge** peer generates personalized recommendations and sends them to the user's smartphone.

3.5 Combination of Multiparty Paradigms

Party prevents unintentional mixing of features while enabling seamless interoperation through placement types. We now discuss how paradigms interoperate without violating individual safety guarantees. Placement types serve as the basic integration mechanism across the paradigms in Party. Each paradigm provides operations on placed values: multitier uses **asLocal** to access remote placed values; choreography uses **comm** to transfer placed values between peers; and collective computations execute in rounds, yielding streams of placed values that are emitted at each round

and can be consumed asynchronously by other paradigms. The **Placement** capability allows placed values to be shared across paradigm boundaries while maintaining (placement) type safety. Finally, the architecture specification constrains which peers can interact, and the type system enforces these constraints regardless of the paradigm that creates or accesses the placed value.

First, we demonstrate the recommendation system, involving *choreographic* and *multitier*. Line 2 declares all required paradigm capabilities via **using**, making their constructs available in the body.

```

1 object RecommendationSystem:
2   def recomm(smartphoneToken: Token on Phone)(using Placement, Choreography, Multitier) =
3     Multitier:
4       val authRequests = on[Edge] { asLocalAll(smartphoneToken) }
5       val authPeers = Choreography:
6         val requests = comm[Edge, Cloud](authRequests)
7         val authGranted = on[Cloud] { /* Grant access based on auth tokens */ }
8         comm[Cloud, Edge](authGranted)
9       val recommendations = on[Edge]:
10        createRecommendations(take(authPeers))
11     on[Phone]:
12       asLocal(recommendations).subscribe: recommendation => /* Display recommendations */

```

Inside the **Choreography** block, **comm**[Edge, Cloud] (Line 6) transfers authentication requests to the **Cloud**. The granted result is then returned to edge nodes (Line 8) and recommendations are delivered to smartphones via **asLocal** (Line 12), crossing paradigm boundaries safely under placement types. In this example, **Choreography** and **Multitier** coexist safely. Choreographic code can transfer values created in multitier code to other peers (Line 6) and multitier code can access the values exchanged in the choreography (Line 10). The snippet shows two explicit cross-paradigm accesses to placed values, from multitier code into choreography and in the opposite direction.

Second, we show the combination of a collective and a multitier component in CityEvt:

```

1 def crowdingLevelAndCloud(using Placement, Collective, Multitier) =
2   val level: Signal[Double] on Phone = Collective[Phone](1.s):
3     /* compute crowding level */
4   Multitier:
5     on[Cloud]:
6       asLocalAll(level).values.foreach: signal =>
7         signal.subscribe: level =>
8           /* monitor crowding level on the cloud */

```

The **crowdingLevelAndCloud** function requires four capabilities (Line 1): **Placement** for placement types, **Collective** for aggregate computing, and **Multitier** for tier-crossing operations. The computation (Line 2) creates a collective scope on the **Phone** peer with 1-second rounds, producing a **Signal**[Double] that continuously emits crowding levels computed by the aggregate program. This signal is placed on the **Phone**, as each smartphone in the system independently computes its local crowding estimate. The **Multitier** scope (Line 4) then accesses the collectively computed values. Within an **on**[Cloud] block (Line 5) the **asLocalAll** operation (Line 6) retrieves the placed signals from all connected smartphones and subscribes to each signal, allowing the cloud to continuously monitor the crowding levels computed by the collective algorithm running on the smartphones.

3.6 Safety Features of Party

Safe Boundaries. In the previous example, the two paradigms are delimited by their respective scopes, and, at their boundary, a placed value is transferred from the collective context to the multitier context through the **asLocalAll** operation, which is defined on the placement types, supported by both paradigms. This integration is safe as the type system checks peer interactions

within paradigm scopes provide the usual guarantees of the paradigm and peer interactions across paradigms only occurs through placed values as well-defined interaction points.

Leak Prevention. In the following code snippet, we exemplify how Party's type system enforces a strict scoping discipline, preventing capabilities to leak outside their intended scope. The example first places a string on **Phone** (Line 2) and then a function also on **Phone** (Line 4), which captures the placed string defined before. The placed function is then sent to the **Cloud** through **comm** (Line 6), where it is invoked (Line 9). Such invocation requires the function to access the captured string from **Phone**. As the function is now on the **Cloud**, however, there is no way to access the string, leading to a runtime error. Party's type system rejects such code at compile time.

```
1 def choreography(using Placement, Choreography): Unit = Choreography:
2   val valueOnSmartphone: String on Phone = on[Phone]:
3     "Hello from Smartphone!"
4   val funAtSmartphone: (() => String) on Phone = on[Phone]:
5     () => take(valueOnSmartphone)
6   val msgAtCloud: (() => String) on Cloud = comm[Phone, Cloud](funAtSmartphone)
7   on[Cloud]:
8     val f: () => String = take(msgAtCloud)
9     f()
```

Deadlock Avoidance. Consider two nested **on** blocks. Operationally, to model placed execution, an **on** block executes its body only when the code is executed for the peer that matches the peer type specified for the **on** block; otherwise, evaluation of the body is skipped – including all nested **on** blocks. If two nested **on** blocks specify different peers, the inner block is never executed because the outer block only runs on its designated peers. This setup can lead to deadlocks if the inner block places a value required by other peers to progress – waiting for a value that is never produced.

In Party, we prevent such deadlocks at compile time. Consider the following example in Party:

```
1 def choreography(using Placement, Choreography): String = Choreography:
2   val msgAtSmartphone: String on Phone = on[Phone]:
3     val onCloud: String on Cloud = on[Cloud]:
4       "Hello from Cloud!"
5     val onSmartphone: String on Phone = comm[Cloud, Phone](onCloud)
6     take(onSmartphone)
```

Line 2 defines a block performing a computation on **Phone**, which contains a computation on the **Cloud** in a nested **on** block (Line 3). Then, the **Cloud**'s value is sent back to the **Phone** through a **comm** operation (Line 5). This code would deadlock since the **Cloud** will never execute the block in Line 3 because it is placed inside a **Phone** block (Line 2), skipping the inner **Cloud** block. Thus, the **Phone** will wait for the value from the **Cloud** that is never produced, leading to a deadlock (Line 5). Party's type system rejects such code at compile time.

4 Technical Realization and Scala 3 Reference Implementation

Our approach leverages three language features related to capabilities. We discuss our reference implementation in Scala 3 – which supports all – but our approach is applicable to other languages (Section 6.3).

First, it requires a mechanism to declare that certain operations require access to specific capabilities. Commonly, capabilities are available in a lexical scope, either explicitly (e.g., as a function parameter) or implicitly (e.g., via contextual binding). Scala 3's contextual abstractions – context parameters (**using** clauses) and context functions (**?=>**) – support modeling capabilities as implicit arguments, i.e., arguments that are transparently synthesized by the compiler at the call side. Using

this mechanism, capabilities do not need to be passed explicitly but can be inferred from the (lexical) context. In Party, each paradigm-specific operation is guarded by a corresponding capability.

Second, our design requires that capabilities cannot escape their lexical scope. Scala 3 provides capture checking [10], which tracks captured capabilities in types, providing a way of preventing them from escaping their scope.

Both the first and the second feature are supported, for example, in languages with algebraic effects, like Koka, which track effects explicitly in function types (cf. Section 6.3.1). Applying an effect then corresponds to invoking an operation and providing an effect handler to binding a capability. The second feature is also supported by techniques to restrict the lifetime of values, e.g., to the scope in which they are introduced, as can be done in languages like Rust (cf. Section 6.3.2).

Third, our design requires certain capabilities not to be shared across multiple use sites simultaneously. Scala 3 features separation checking [68] to prevent capabilities from being aliased, preventing unintended sharing of capabilities. The combination of both capture and separation checking prevents capabilities from leaking, e.g., ensuring that computations intended for a specific peer do not accidentally capture values or capabilities that are invalid in other peer contexts.

The third feature can also be supported by *borrow-checking* the usage of values, as supported, for example, in Rust (cf. Section 6.3.2).

4.1 Implementation of Safety Features

Party leverages capabilities with capture and separation checking to provide complementary safety guarantees for distributed programming.

Context Functions. To enforce that each multiparty operation can only be used within its respective context, in Party, a context function delimits the scope of each paradigm and provides the necessary capability for its operations inside its scope:

```
1 trait Multiparty extends SharedCapability
2 trait Scope[L] extends SharedCapability
```

Multiparty is the base capability for using multiparty programming paradigms while **Scope[L]** models the capability of a specific multiparty paradigm provided for a lexical scope. The companion object of each capability provides the definition of the `apply` function as the entry point for opening a scope of the respective paradigm. Consider the example of the **Multitier** capability:

```
1 object Multitier:
2   sealed trait MultitierScope extends Erased
3
4   def asLocal[L <: TiedSingleWith[R], R <: Peer: PeerTag, V, SC](
5     using Multitier, PeerScope[L], Scope[SC], SC :=> MultitierScope)(
6     value: V on R): V
7
8   def apply[A](using Multitier)(multitier: Scope[MultitierScope] ?=> A): A
9     ... // Other operations
```

Line 2 defines a marker trait for the multitier scope. Erased traits are solely used for type-level tracking and have no runtime representation. The operations of the **Multitier** capability require a capability **Scope[SC]** along with the evidence that SC equals **MultitierScope** (Line 5), which ensures that operations can only be invoked if the innermost **Scope** is multitier. The only way to obtain such **Scope** is via the `apply` method (Line 8), which supplies a **Scope[MultitierScope]** capability (via a context function `?=>`) to the code block for the multitier scope. Further, `apply` ensures that the multitier scope can only be created when the **Multitier** capability is available (Line 8). The **Collective** and **Choreography** capabilities have an analogous structure.

Capture Checking. Statically tracking capabilities via capture checking has two roles. First, it prevents leaking the capability outside the intended paradigm context. By defining the **Scope**[L] as a (shared) capability, we enable its tracking through the Scala capture checking mechanism, ensuring that it cannot leak outside the multitier thunk passed to the apply method (Line 8). If we try to return the **Scope**[**MultitierScope**] from the apply function, the type system triggers a capture error, preventing the scope from escaping and being used outside the multitier context.

Second, expression placement is tracked as well via the **Placement** capability, for example, to avoid calling a function that captures a peer-specific capability from another peer (cf. Section 3.6). Capture checking prevents such unsafe patterns.

Separation Checking. Separation checking distinguishes *read-only* capabilities – which can be freely shared and aliased – and capabilities with side effects (e.g., mutate state), which cannot. In our implementation, the **Placement** capability is defined as a **Mutable** capability, preventing **Placement** from being aliased, avoiding deadlocks due to nested **on** blocks, as the inner block would need access to the same **Placement** capability reference. A detailed example of the issue of nesting **on** blocks can be found in Section 6.3. The **Placement** capability is mutable because invoking its **on** method mutates the capability by requiring it to produce a fresh identifier for each placed value:

```
1 trait Placement extends Mutable:
2   update def on[P <: Peer][V](body: PeerScope[P] ?=> V): V on P
```

5 Formalization

In this section we formalize a core calculus for Party, which models the integration of the *collective*, *multitier* and *choreography* paradigms with the help of *placement types* and *capabilities*.

5.1 Placement

Definition 5.1 shows the syntax of the core calculus with placement types and placement capability.

Expressions. The expression language extends the simply-typed lambda calculus (variables, applications, abstractions) with *placement* expressions **on** and the **take** operation to access local placed values. The **handle** construct serves as an execution entry point by providing a specified capability. Values encompass lambda abstractions, placed values (values annotated with their peer location), and basic values including units, booleans, and maps.

Capabilities. Function types are annotated with capabilities. Invoking a function with a non-empty capability set C incurs the effects associated with the specified capabilities. Thus, a function with an empty capability set denotes a pure function without side effects. Capabilities govern access to operations: Pl for placement, M for multitier (Section 5.2), Cl for collective (Section 5.3), and Ch for choreographies (Section 5.4). Peer capabilities (Ps P) restrict operations to specific peer types.

To provide a Scala 3 analogy, the capability-annotated function type $T_1 \rightarrow_C T_2$ is a function carrying a set of capabilities denoted by C, while $T_1 \rightarrow T_2$ corresponds to pure function, i.e., a function without any capability references in its capture set.

Peers and Placement Types. Peers P correspond to the peer types declared in the architecture specification, representing distinct computational nodes. Placement types U associate types with their designated peer location, enabling static verification of placement constraints.

Definition 5.1 (Syntax).

Expressions	$e ::= x \mid e e \mid \lambda x : T. e \mid \text{handle } C \ P \ e \mid$
Placement Expressions	$\text{on } P \ e \mid \text{take } e$
Values	$v ::= \lambda x : T. e \mid v \text{ on } P \mid \text{unit} \mid \text{true} \mid \text{false} \mid \text{Map ID } v$
Types	$T ::= T \rightarrow_C T \mid \text{Map ID } T \mid \text{Bool} \mid \text{unit} \mid U$
Capabilities	$C ::= \text{Pl} \mid \text{Scope } C \mid \text{Ps } P$
Peers	$P ::= \emptyset \mid P, p$
Placement Types	$U ::= P \text{ on } T$

Dynamic Semantics. The operational semantics defines where expressions evaluate in the distributed system. To preserve the single global perspective of multiparty programming, we model the evaluation as a single thread of execution annotating the reduction relation with a set of peer instances π^P on which an evaluation step takes place, i.e., the reduction step takes place on all peer instances in π^P simultaneously, instead of modeling all peer instances separately as asynchronous processes. This idea is similar to how global program execution has been modeled in multitier languages previously [65, 69]. Our evaluation judgments take the form $\Omega; \sigma \triangleright e \rightarrow^{\pi^P} \Omega'; \sigma' \triangleright e'$: Under peer configuration Ω and VM state σ , expression e evaluates to e' with updated environment Ω' and state σ' , executing on the peer instances π^P . We will discuss the VM state in Section 5.3 because the reduction rules described elsewhere do not change the VM state.

Definition 5.2 (Peer Configuration). A peer configuration Ω captures the runtime network topology by mapping peer types to their active device instances: $\Omega ::= \emptyset \mid \Omega, P : \{\pi_1^P, \dots, \pi_n^P\}$

Here $P : \{\pi_1^P, \dots, \pi_n^P\}$ indicates that peer type P currently has peer instances $\pi_1^P, \dots, \pi_n^P$ active in the network.

Definition 5.3 (Peer Connectivity). We support two types of peer relationships that govern data flow and communication:

$$\begin{aligned} P_s &\mapsto^1 P_t && \text{each device in } P_s \text{ connects to exactly one device in } P_t \\ P_s &\mapsto^* P_t && \text{each device in } P_s \text{ can connect to multiple devices in } P_t \end{aligned}$$

These relationships are mutually exclusive and remain stable during individual evaluation steps. The evaluation rules are in Definition 5.4. **E-AppE1**, **E-AppE2**, and **E-App** are standard. **E-One** and **E-OnV** manage placement expressions, enabling computations to be performed at specified peer locations within the network topology. **E-TakeE** and **E-TakeV** enable reading placed values on the local peer on which they are placed. **E-Handle** and **E-HandleV** manage capability-controlled execution, ensuring that the program runs within its required capabilities.

Definition 5.4 (Evaluation Rules).

$$\begin{aligned} \text{(E-AppE1)} \quad & \frac{\Omega; \sigma \triangleright e_1 \rightarrow^{\pi^P} \Omega; \sigma \triangleright e_1'}{\Omega; \sigma \triangleright e_1 e_2 \rightarrow^{\pi^P} \Omega; \sigma \triangleright e_1' e_2'} & \text{(E-AppE2)} \quad & \frac{\Omega; \sigma \triangleright e_2 \rightarrow^{\pi^P} \Omega; \sigma \triangleright e_2'}{\Omega; \sigma \triangleright v e_2 \rightarrow^{\pi^P} \Omega; \sigma \triangleright v e_2'} \\ \text{(E-App)} \quad & \Omega; \sigma \triangleright (\lambda x : T. e) v \rightarrow^{\pi} \Omega; \sigma \triangleright [v/x]e \\ \text{(E-One)} \quad & \frac{\Omega; \sigma \triangleright e \rightarrow^{\pi^P} \Omega; \sigma \triangleright e'}{\Omega; \sigma \triangleright \text{on } P \ e \rightarrow^{\pi^P} \Omega; \sigma \triangleright \text{on } P \ e'} & \text{(E-OnV)} \quad & \Omega; \sigma \triangleright \text{on } P \ v \rightarrow^{\pi^P} \Omega; \sigma' \triangleright v \text{ on } P \\ \text{(E-TakeE)} \quad & \frac{\Omega; \sigma \triangleright e \rightarrow^{\pi^P} \Omega; \sigma \triangleright e'}{\Omega; \sigma \triangleright \text{take } e \rightarrow^{\pi^P} \Omega; \sigma \triangleright \text{take } e'} & \text{(E-TakeV)} \quad & \Omega; \sigma \triangleright \text{take } v \text{ on } P \rightarrow^{\pi^P} \Omega; \sigma \triangleright v \\ \text{(E-Handle)} \quad & \frac{\Omega; \sigma \triangleright e \rightarrow^{\pi^P} \Omega; \sigma \triangleright e'}{\Omega; \sigma \triangleright \text{handle } C \ P \ e \rightarrow^{\pi^P} \Omega; \sigma \triangleright e'} & \text{(E-HandleV)} \quad & \Omega; \sigma \triangleright \text{handle } C \ P \ v \rightarrow^{\pi^P} \Omega; \sigma \triangleright v \end{aligned}$$

Static Semantics. The typing judgments take the form $\mathcal{A}, \Gamma \vdash e : T$ with C : Under the architecture $\mathcal{A}$ and variable environment Γ , expression e has type T and requires the set of capabilities C . To provide a Scala 3 analogy, this is equivalent to a context function with context parameters representing the capabilities. The architecture $\mathcal{A}$ captures the architectural specification through peer and tie definitions, while Γ maintains the type bindings for variables in scope.

The structure of the typing contexts is in [Definition 5.5](#). An architecture combines both the collection of peer types and the ties for their interactions. We require that every peer referenced in a tie relationship is explicitly declared in the peer type collection. Tie relationships come in two flavors: single ties $P_s \mapsto^1 P_t$ establish a one-to-one connection between source and target peers, while multiple ties $P_s \mapsto^* P_t$ allow a source peer to connect to several target peers. The typing context Γ follows standard conventions, starting empty and growing with variable-type bindings.

Definition 5.5 (Typing Context).

Architecture $\mathcal{A} ::= \{P\}; \{P_1 \mapsto^1 P_2\}; \{P_3 \mapsto^* P_4\}$ Typing Context $\Gamma ::= \emptyset \mid \Gamma, x : T$

The typing rules are in [Definition 5.6](#). **T-VAR** types variables by looking up their type in the context Γ . **T-ABS** types lambda abstractions: the body e is typed under an extended context with the bound variable $x : T_1$. The capabilities required by the abstraction itself (C) are those of the body minus the capabilities captured by the function type (C_1). **T-APP** types function application by combining the capabilities required by the function, the argument, and the capabilities declared in the function type. **T-ON** types placement expressions on P e , which place computations on a specific peer type P , requiring the placement capability PI and the $P_s P$ capability parametrized over the peer. The rule ensures that peer capabilities are consumed (not propagated outward), enforcing that peer-specific operations remain local to their designated peer. **T-HANDLE** types the handle $C P e$ construct, which injects a capability C into the evaluation context of e . The body e is typed with the required capability C , and the overall expression discharges this capability, requiring no additional capabilities ($\emptyset$) from the outer term.

Definition 5.6 (Typing).

$$\begin{array}{c}
 \text{(T-VAR)} \quad \frac{x : T \in \Gamma}{\mathcal{A}; \Gamma \vdash x : T \text{ with } \emptyset} \qquad \text{(T-ABS)} \quad \frac{\mathcal{A}; \Gamma, x : T_1 \vdash e : T_2 \text{ with } C' \quad C' = C_1 \cup C}{\mathcal{A}; \Gamma \vdash \lambda x : T_1. e : T_1 \rightarrow_{C_1} T_2 \text{ with } C} \\
 \text{(T-APP)} \quad \frac{\mathcal{A}; \Gamma \vdash e_1 : T_1 \rightarrow_{C_1} T_2 \text{ with } C_2 \quad \mathcal{A}; \Gamma \vdash e_2 : T_1 \text{ with } C_3}{\mathcal{A}; \Gamma \vdash e_1 e_2 : T_2 \text{ with } C_1 \cup C_2 \cup C_3} \\
 \text{(T-ON)} \quad \frac{\mathcal{A}; \Gamma \vdash e : T \text{ with } C \cup P_s P \setminus PI}{\mathcal{A}; \Gamma \vdash \text{on } P e : T \text{ on } P \text{ with } C \cup PI} \qquad \text{(T-TAKE)} \quad \frac{\mathcal{A}; \Gamma \vdash e : T \text{ on } P \text{ with } C \cup P_s P}{\mathcal{A}; \Gamma \vdash \text{take } e : T \text{ with } C \cup P_s P} \\
 \text{(T-HANDLE)} \quad \frac{\mathcal{A}; \Gamma \vdash e : T \text{ with } C \cup C_1}{\mathcal{A}; \Gamma \vdash \text{handle } C P e : T \text{ with } C_1}
 \end{array}$$

5.2 Multitier

[Definition 5.7](#) presents the syntax additions of the multitier paradigm. The multitier capability M governs access to the multitier operations **asLocal** and **asLocalAll** for tier-crossing computations.

Definition 5.7 (Syntax).

Multitier Expressions $e ::= \dots \mid \text{asLocal } e \mid \text{asLocalAll } e$
 Capabilities $C ::= \dots \mid M$

Dynamic Semantics. The reduction rules for multitier programming are in [Definition 5.8](#). [E-AsLOCALe](#) and [E-AsLOCALv](#) allow values placed on one peer to be accessed on another peer, respecting single tie constraints. [E-AsLOCALAllE](#) and [E-AsLOCALAllV](#) handle multiple ties where one peer aggregates values from multiple remote peers simultaneously.

Definition 5.8 (Evaluation Rules).

$$\begin{array}{c}
 \text{(E-AsLOCALe)} \quad \frac{\Omega; \sigma \triangleright e \rightarrow \pi^{P_r} \quad \Omega; \sigma \triangleright e'}{\Omega; \sigma \triangleright \text{asLocal } e \text{ on } P_r \rightarrow \pi^{P_r} \quad \Omega; \sigma \triangleright \text{asLocal } e' \text{ on } P_r} \\
 \text{(E-AsLOCALv)} \quad \Omega; \sigma \triangleright \text{asLocal } v \text{ on } P_r \rightarrow \pi^{P_l} \quad \Omega; \sigma \triangleright v \\
 \text{(E-AsLOCALAllE)} \quad \frac{\Omega; \sigma \triangleright e \rightarrow \pi^{P_r} \quad \Omega; \sigma \triangleright e'}{\Omega; \sigma \triangleright \text{asLocalAll } e \text{ on } P_r \rightarrow \pi^{P_r} \quad \Omega; \sigma \triangleright \text{asLocalAll } e' \text{ on } P_r} \\
 \text{(E-AsLOCALAllV)} \quad \Omega; \sigma \triangleright \text{asLocalAll } v \text{ on } P_r \rightarrow \pi^{P_l} \quad \Omega; \sigma \triangleright \{ \pi^{P_r} \mapsto v \}
 \end{array}$$

Static Semantics. The typing rules for multitier programming are in [Definition 5.9](#). [T-AsLOCAL](#) enables accessing a remotely placed value. Given an expression of type T on P_r the rule produces a value of type T on the local peer P_l , provided the tie constraint $P_l \mapsto^1 P_r$ holds. This operation requires multitier (M), and local peer (Ps P_l) capabilities. [T-AsLOCALAll](#) handles the case where the local peer is tied to multiple remote peers ($P_l \mapsto^* P_r$). The result is a map from device identifiers to values, collecting responses from all reachable remote peers.

Definition 5.9 (Typing).

$$\begin{array}{c}
 \text{(T-AsLOCAL)} \quad \frac{\mathcal{A}; \Gamma \vdash e : T \text{ on } P_r \text{ with } C \cup \{M, Ps \ P_l, \text{Scope } M\} \quad P_l \mapsto^1 P_r \in \mathcal{A}}{\mathcal{A}; \Gamma \vdash \text{asLocal } e : T \text{ with } C \cup \{M, Ps \ P_l, \text{Scope } M\}} \\
 \text{(T-AsLOCALAll)} \quad \frac{\mathcal{A}; \Gamma \vdash e : T \text{ on } P_r \text{ with } C \cup \{M, Ps \ P_l, \text{Scope } M\} \quad P_l \mapsto^* P_r \in \mathcal{A}}{\mathcal{A}; \Gamma \vdash \text{asLocalAll } e : \text{Map ID } T \text{ with } C \cup \{M, Ps \ P_l, \text{Scope } M\}}
 \end{array}$$

5.3 Aggregate

[Definition 5.10](#) presents the syntax additions of the aggregate paradigm. The aggregate capability Cl governs access to the aggregate operations **collective**, **rep**, **nbr**, and **branch**.

Definition 5.10 (Syntax).

$$\begin{array}{ll}
 \text{Collective Expressions} & e ::= \dots \mid \text{collective } e \mid \text{rep } e \ e \mid \text{nbr } e \mid \text{branch } e \ e \ e \\
 \text{Capabilities} & C ::= \dots \mid Cl
 \end{array}$$

Dynamic Semantics. The rules for aggregate computing access the state of the aggregate VM.

Definition 5.11 (VM State and Device Context). The VM state σ maintains persistent information across computation rounds for collective operations: $\sigma : \text{Loc} \rightarrow v$

The VM state is used by aggregate operators (**rep**, **nbr**, **branch**) to store intermediate values, neighbor information, and branch conditions necessary for distributed coordination. We write $\sigma[\theta \leftarrow v]$ for state updates (indicating that location θ is updated to store value v) and $\sigma(\theta)$ for state access at location θ . We use $\theta = \text{fresh}$ to denote the generation of a unique location $\theta \in \text{Loc}$ that is not already present in the current VM state σ , i.e., $\theta \notin \text{dom}(\sigma)$.

The reduction rules for aggregate computing are in [Definition 5.12](#). [E-COLLE](#) and [E-COLLV](#) execute aggregate computations that span across network peers. For stateful collective operations, the VM state σ maintains persistence across computation rounds: [E-REPE1](#) and [E-REPE2](#) handle **rep**, initializing and updating state values iteratively; [E-NBRE](#) and [E-NBRV](#) handle **nbr**, creating a field by looking up in the σ the most recent neighbor values produced at the point identified by the location θ ; [E-BRANCHE](#), [E-BRANCHVTRUE](#), and [E-BRANCHVFALSE](#) handle **branch**, storing branch conditions in VM state to ensure domain alignment across the network.

Definition 5.12 (Evaluation Rules).

$$\begin{array}{c}
\text{(E-COLL E)} \frac{\Omega; \sigma \triangleright e \rightarrow^{\pi^P} \Omega; \sigma \triangleright e'}{\Omega; \sigma \triangleright \text{collective } e \rightarrow^{\pi^P} \Omega; \sigma \triangleright \text{collective } e'} \quad \text{(E-COLL V)} \frac{}{\Omega; \sigma \triangleright \text{collective } v \rightarrow^{\pi^P} \Omega; \sigma' \triangleright v \text{ on } P} \\
\\
\text{(E-REP E)} \frac{\Omega; \sigma \triangleright e_1 \rightarrow^{\pi^P} \Omega; \sigma \triangleright e_1'}{\Omega; \sigma \triangleright \text{rep } e_1 \ e_2 \rightarrow^{\pi^P} \Omega; \sigma \triangleright \text{rep } e_1' \ e_2} \quad \text{(E-REP E2)} \frac{\theta = \text{fresh}}{\Omega; \sigma \triangleright \text{rep } v \ e_2 \rightarrow^{\pi^P} \Omega; \sigma[\theta \leftarrow v] \triangleright e_2 \ v} \\
\\
\text{(E-NBR E)} \frac{\Omega; \sigma \triangleright e \rightarrow^{\pi^P} \Omega; \sigma \triangleright e'}{\Omega; \sigma \triangleright \text{nbr } e \rightarrow^{\pi^P} \Omega; \sigma \triangleright \text{nbr } e'} \quad \text{(E-NBR V)} \frac{\theta = \text{fresh} \quad \forall \pi \in \pi^P. \sigma(\theta) = v_\pi}{\Omega; \sigma \triangleright \text{nbr } v \rightarrow^{\pi^P} \Omega; \sigma[\theta \leftarrow v] \triangleright v \ [\pi \mapsto v_\pi]} \\
\\
\text{(E-BRANCH E)} \frac{\Omega; \sigma \triangleright e_1 \rightarrow^{\pi^P} \Omega; \sigma \triangleright e_1'}{\Omega; \sigma \triangleright \text{branch } e_1 \ e_2 \ e_3 \rightarrow^{\pi^P} \Omega; \sigma \triangleright \text{branch } e_1' \ e_2 \ e_3} \\
\\
\text{(E-BRANCH VTRUE)} \frac{\theta = \text{fresh}}{\Omega; \sigma \triangleright \text{branch true } e_2 \ e_3 \rightarrow^{\pi^P} \Omega; \sigma[\theta \leftarrow \text{true}] \triangleright e_2} \\
\\
\text{(E-BRANCH VFALSE)} \frac{\theta = \text{fresh}}{\Omega; \sigma \triangleright \text{branch false } e_2 \ e_3 \rightarrow^{\pi^P} \Omega; \sigma[\theta \leftarrow \text{false}] \triangleright e_3}
\end{array}$$

Static Semantics. The typing rules for aggregate programming are in [Definition 5.13](#). **T-COLLECTIVE** types aggregate expressions that execute collectively across peers P . The body must be a function requiring the peer capability $\text{Ps } P$ and access to the virtual machine state Vm . The tie constraint $P \mapsto^* P$ reflects that devices of the same peer type can communicate with each other. The result is a placed value T on P , and the operation requires collective (Cl) and peer scope (Ps P) capabilities. **T-REP** types the repetition construct, maintaining a state across computation rounds. It takes an initial value e_1 of type T and an update function e_2 of type $T \rightarrow T$, both requiring collective and virtual machine capabilities. **T-NBR** types neighbor value exchange, allowing devices to share values with their neighbors. **T-BRANCH** types conditional branching in aggregate computations, ensuring both branches have the same type and capability requirements.

Definition 5.13 (Typing).

$$\begin{array}{c}
\text{(T-COLLECTIVE)} \frac{\mathcal{A}; \Gamma \vdash e : T_1 \rightarrow \{\text{Vm}\} \ T_2 \text{ with Cl} \quad P \mapsto^* P \in \mathcal{A}}{\mathcal{A}; \Gamma \vdash \text{collective } e : T_2 \text{ on } P \text{ with } C \cup \{\text{Cl}, \text{Ps } P\}} \quad \text{(T-REP)} \frac{\mathcal{A}; \Gamma \vdash e_1 : T \text{ with } \{\text{Cl}, \text{Vm}\} \quad \mathcal{A}; \Gamma \vdash e_2 : T \rightarrow T \text{ with } \{\text{Cl}, \text{Vm}\}}{\mathcal{A}; \Gamma \vdash \text{rep } e_1 \ e_2 : T \text{ with } \{\text{Cl}, \text{Vm}\}} \\
\\
\text{(T-BRANCH)} \frac{\mathcal{A}; \Gamma \vdash e_{\text{cond}} : \text{Bool with } \{\text{Cl}, \text{Vm}\} \quad \mathcal{A}; \Gamma \vdash e_{\text{th}} : T \text{ with } \{\text{Cl}, \text{Vm}\} \quad \mathcal{A}; \Gamma \vdash e_{\text{el}} : T \text{ with } \{\text{Cl}, \text{Vm}\}}{\mathcal{A}; \Gamma \vdash \text{branch } e_{\text{cond}} \ e_{\text{th}} \ e_{\text{el}} : T \text{ with } \{\text{Cl}, \text{Vm}\}} \quad \text{(T-NBR)} \frac{\mathcal{A}; \Gamma \vdash e : T \text{ with } \{\text{Cl}, \text{Vm}\}}{\mathcal{A}; \Gamma \vdash \text{nbr } e : \text{Map ID } T \text{ with } \{\text{Cl}, \text{Vm}\}}
\end{array}$$

5.4 Choreography

[Definition 5.14](#) presents the syntax additions of the choreography paradigm. The choreography capability Ch governs access to the choreography operations **comm**, **multicast** and **broadcast**.

Definition 5.14 (Syntax).

$$\begin{array}{ll}
\text{Choreography Expressions} & e ::= \dots \mid \text{comm } e \mid \text{multicast } e \mid \text{broadcast } e \\
\text{Capabilities} & C ::= \dots \mid \text{Ch}
\end{array}$$

Dynamic Semantics. The reduction rules for choreographic programming are in [Definition 5.15](#). **E-COMME** and **E-COMMV** facilitate inter-peer communication by transferring placed values between connected peers. **E-MCASTE** and **E-MCASTV** extend communication to one-to-many scenarios, where a placed value from a source peer is transmitted to multiple target peers connected via multiple ties, producing a placed value on each receiving peer. **E-BCASTE** and **E-BCASTV** enable network-wide broadcasting, distributing a value to all peers in the network regardless of their type, returning the plain value on each peer without placement annotations.

Definition 5.15 (Evaluation Rules).

$$\begin{array}{c}
\text{(E-COMME)} \frac{\Omega; \sigma \triangleright e \rightarrow^{\pi^{P_s}} \Omega; \sigma \triangleright e'}{\Omega; \sigma \triangleright \text{comm } e \rightarrow^{\pi^{P_s}} \Omega; \sigma \triangleright \text{comm } e'} \quad \text{(E-COMMV)} \frac{P_s \mapsto^1 P_r \in \mathcal{A}}{\Omega; \sigma \triangleright \text{comm } v \text{ on } P_s \rightarrow^{\pi^{P_r}} \Omega; \sigma' \triangleright v \text{ on } P_r} \\
\text{(E-MCASTE)} \frac{\Omega; \sigma \triangleright e \rightarrow^{\pi^{P_s}} \Omega; \sigma \triangleright e'}{\Omega; \sigma \triangleright \text{multicast } e \rightarrow^{\pi^{P_s}} \Omega; \sigma \triangleright \text{multicast } e'} \quad \text{(E-MCASTV)} \frac{P_s \mapsto^* P_r \in \mathcal{A} \quad \forall \pi_i^{P_r} \in \Omega(P_r)}{\Omega; \sigma \triangleright \text{multicast } v \text{ on } P_s \rightarrow^{\pi_i^{P_r}} \Omega; \sigma' \triangleright v \text{ on } P_r} \\
\text{(E-BCASTE)} \frac{\Omega; \sigma \triangleright e \rightarrow^{\pi^P} \Omega; \sigma \triangleright e'}{\Omega; \sigma \triangleright \text{broadcast } e \rightarrow^{\pi^P} \Omega; \sigma \triangleright \text{broadcast } e'} \quad \text{(E-BCASTV)} \frac{\forall P \in \mathcal{A}, \forall \pi_i^P \in \Omega(P)}{\Omega; \sigma \triangleright \text{broadcast } v \rightarrow^{\pi_i^P} \Omega; \sigma' \triangleright v}
\end{array}$$

Static Semantics. The typing rules for choreographic programming are in [Definition 5.16](#). **T-COMM** types choreographic communication, transferring a placed value from sender peer P_s to receiver peer P_r . The tie constraint $P_s \mapsto^1 P_r \in \mathcal{A}$ ensures that a valid communication channel exists between the peers, requiring both choreography (Ch) and placement (Pl) capabilities. **T-MCAST** handles choreographic communication in one-to-many scenarios. The tie constraint $P_s \mapsto^* P_r \in \mathcal{A}$ ensures that the sender can reach multiple target peers. Like **T-COMM**, it requires choreography and placement capabilities, and produces a placed value on each receiving peer. **T-BCAST** types network-wide broadcasting, which distributes a value to all peers in the network. Unlike the other communication rules, broadcasting returns the plain type T without placement annotations, as the value is made available on all peers. It requires choreography and network capabilities.

Definition 5.16 (Typing).

$$\begin{array}{c}
\text{(T-COMM)} \frac{\mathcal{A}; \Gamma \vdash e : T \text{ on } P_s \text{ with } C \cup \{\text{Ch, Pl, Scope Ch}\} \quad P_s \mapsto^1 P_r \in \mathcal{A}}{\mathcal{A}; \Gamma \vdash \text{comm } e : T \text{ on } P_r \text{ with } C \cup \{\text{Ch, Pl, Scope Ch}\}} \\
\text{(T-MCAST)} \frac{\mathcal{A}; \Gamma \vdash e : T \text{ on } P_s \text{ with } C \cup \{\text{Ch, Pl, Scope Ch}\} \quad P_s \mapsto^* P_r \in \mathcal{A}}{\mathcal{A}; \Gamma \vdash \text{multicast } e : T \text{ on } P_r \text{ with } C \cup \{\text{Ch, Pl, Scope Ch}\}} \\
\text{(T-BCAST)} \frac{\mathcal{A}; \Gamma \vdash e : T \text{ with } C \cup \{\text{Ch, Pl, Scope Ch}\}}{\mathcal{A}; \Gamma \vdash \text{broadcast } e : T \text{ with } C \cup \{\text{Ch, Pl, Scope Ch}\}}
\end{array}$$

5.5 Type Soundness

In this section, we prove the calculus correct. The first property is type soundness, ensuring that well-typed programs do not get stuck during evaluation. This property is guaranteed given the following well-formedness conditions for peer configurations.

5.5.1 Well-formedness Conditions. We establish the conditions under which our distributed calculus operates correctly and safely.

Definition 5.17 (Architecture-Satisfying Peer Configuration). A peer configuration Ω satisfies an architecture $\mathcal{A}$, written $\Omega \models \mathcal{A}$ if:

- (1) $\forall \pi_i^P \in \Omega. P \in \mathcal{A}$ (peer coverage)
- (2) $\forall P_s \mapsto^1 P_t \in \mathcal{A}, \forall \pi_i^{P_s} \in \Omega. |\{ \pi_j^{P_t} \in \Omega \mid \pi_i^{P_s} \rightsquigarrow \pi_j^{P_t} \}| = 1$ (single tie)

$$(3) \forall P_s \mapsto^* P_t \in \mathcal{A}, \forall \pi_i^{P_s} \in \Omega. \left| \{ \pi_j^{P_t} \in \Omega \mid \pi_i^{P_s} \rightsquigarrow \pi_j^{P_t} \} \right| \geq 0 \quad (\text{multi tie})$$

where $\pi_i^{P_s} \rightsquigarrow \pi_j^{P_t}$ denotes runtime connectivity.

Definition 5.18 (VM State Consistency). Recall that the VM state $\sigma : \text{Loc} \rightarrow \text{v}$ is a partial function mapping locations to values, where every $\theta \in \text{dom}(\sigma)$ is allocated by a collective construct (rep, nbr, or branch) via $\theta = \text{fresh}$. Let C denote the set of collective construct occurrences in expression e , and let $\text{loc} : C \rightarrow \text{Loc}$ be the injective mapping from each construct occurrence to its allocated location. A VM state σ is *consistent* with respect to a typing derivation $\mathcal{A}, \Gamma \vdash e : T$ with C if the following conditions hold:

- (1) **Domain coverage.** $\text{dom}(\sigma) \subseteq \text{im}(\text{loc})$, i.e., every location in σ was allocated by some collective construct in e .
- (2) **Well-typedness.** For every $c \in C$ with $\theta = \text{loc}(c)$ and $\theta \in \text{dom}(\sigma)$:

$$\sigma(\theta) = v \implies \mathcal{A}, \Gamma \vdash v : \text{ty}(c) \text{ with } \emptyset$$

where $\text{ty}(c)$ is the *stored type* of construct c , defined as:

$$\text{ty}(c) = \begin{cases} T & \text{if } c = \text{rep } e_1 \ e_2 \text{ and } \mathcal{A}, \Gamma \vdash \text{rep } e_1 \ e_2 : T \text{ with } C \\ T & \text{if } c = \text{nbr } e' \text{ and } \mathcal{A}, \Gamma \vdash \text{nbr } e' : \text{Map ID } T \text{ with } C \\ \text{Bool} & \text{if } c = \text{branch } e_{\text{cond}} \ e_{\text{th}} \ e_{\text{el}} \end{cases}$$

- (3) **Injectivity.** The function loc is injective: distinct construct occurrences are mapped to distinct locations, i.e., $\forall c_1, c_2 \in C. c_1 \neq c_2 \implies \text{loc}(c_1) \neq \text{loc}(c_2)$.

5.5.2 Progress and Preservation. The standard progress and preservation properties hold under our defined well-formedness conditions.

THEOREM 5.19 (PROGRESS). *For an architecture-satisfying peer configuration $\Omega \models \mathcal{A}$ with consistent VM states, If π_i^P is a peer instance in Ω with expression e such that $\mathcal{A}, \Gamma \vdash e : T$ with C , then either e is a value or $\Omega; \sigma \triangleright e \rightarrow^{\pi_i^P} \Omega'; \sigma' \triangleright e'$ with consistent VM states.*

PROOF. By induction on the typing derivation. See [Appendix B](#) for the detailed proof. $\square$

THEOREM 5.20 (PRESERVATION). *For an architecture-satisfying peer configuration $\Omega \models \mathcal{A}$ with consistent VM states, if $\mathcal{A}, \Gamma \vdash e : T$ with C for some expression e on peer instance π_i^P , and $\Omega; \sigma \triangleright e \rightarrow^{\pi_i^P} \Omega'; \sigma' \triangleright e'$, then $\mathcal{A}, \Gamma \vdash e' : T$ with C with consistent VM states.*

PROOF. By induction on the reduction rules. See [Appendix B](#) for the detailed proof. $\square$

LEMMA 5.21 (ARCHITECTURE CONFORMANCE). *For an architecture-satisfying peer configuration $\Omega \models \mathcal{A}$ with consistent VM states, then for every evaluation step $\Omega; \sigma \triangleright e \rightarrow^{\pi_i^P} \Omega'; \sigma' \triangleright e'$ where e is one of $\text{comm } v$ on P_l , $\text{asLocal } v$ on P_r , or $\text{asLocalAll } v$ on P_r , the tie $P_s \mapsto P_t \in \mathcal{A}$ holds.*

PROOF. Immediate from the typing rules **T-COMM**, **T-ASLOCAL**, and **T-ASLOCALALL**, which require the corresponding tie constraints in $\mathcal{A}$ as premises, and from the evaluation rules **E-COMMV**, **E-ASLOCALV**, and **E-ASLOCALALLV**, which enforce these constraints at reduction time. $\square$

LEMMA 5.22 (INTER-PARADIGM SAFETY). *For an architecture-satisfying peer configuration $\Omega \models \mathcal{A}$ with consistent VM states, then for every peer instance π_i^P with expression e such that $\mathcal{A}, \Gamma \vdash e : T$ with C , either e is a value, or there exist Ω', σ', e' such that $\Omega; \sigma \triangleright e \rightarrow^{\pi_i^P} \Omega'; \sigma' \triangleright e'$ and $\mathcal{A}, \Gamma' \vdash e' : T$ with C' for some $\Gamma' \supseteq \Gamma$ and $C' \subseteq C$.*

PROOF. Follows directly from [Theorems 5.19](#) and [5.20](#). Progress guarantees that well-typed expressions are either values or can step; preservation ensures the resulting expression remains well-typed. The capability-based separation of choreographic (Ch), multitier (M), and collective (Cl) constructs prevents ill-formed cross-paradigm interactions by construction. $\square$

6 Evaluation

In this section, we evaluate Party, considering three research questions:

RQ1: *Can Party’s capability-based design express choreographic, multitier, and aggregate abstractions?*

[Section 6.1](#) assesses the expressiveness and safety of Party in capturing key programming abstractions from choreographic, multitier, and aggregate paradigms.

RQ2: *Can Party integrate these paradigms in a single distributed program?*

[Section 6.2](#) evaluates how Party safely integrates these paradigms within a single program, ensuring that inter-paradigm interactions do not lead to runtime errors.

RQ3: *Is Party’s approach of supporting choreographic, multitier, and aggregate programming via capabilities generally applicable?*

[Section 6.3](#) investigates the portability of Party’s safety features, exploring how to implement similar safety guarantees in other languages than Scala 3 showing that Party’s approach is broadly applicable.

6.1 Expressivity and Safety

We collected 46 use cases from the literature on multitier programming, choreographies, and aggregate computing. The case studies belong to different domains: communication, coordination, security, games, data and services, algorithms and computation, and spatial applications. [Table 1](#) summarizes these use cases and the programming paradigms they use. To answer [RQ 1](#), we reimplemented each use case in Party. In the companion artifact, we provide the full implementations side-by-side with the original version.

6.1.1 Expressivity. Overall, our implementation comprises 3,874 LoC of Party code, broken down into 2,012 LoC for choreography use cases, 1,425 LoC for multitier use cases, and 437 LoC for aggregate use cases. Our case studies show that Party is expressive enough to support the core abstractions of all three paradigms considered in this work across all the domains in the use cases. We successfully encoded representative multitier, choreographic, and aggregate applications in isolation, demonstrating that Party can capture the essential constructs of each paradigm. This indicates that Party provides a unified approach in

Table 1. Summary of Use Cases in Party.

Class	Use Case	Parad.
Communic.	Basic Chat [56]	Multi.
	Chat Application [65]	Multi.
	Chirper [56]	Multi.
	Chirper Inc [56]	Multi.
	Haste Chatbox* [29]	Multi.
	Email Application* [38]	Chor.
	Email System* [65]	Multi.
	Vitals Streaming [37]	Chor.
	Ping-Pong [6]	Chor.
	Multicast [6]	Chor.
	Broadcasting [18]	Aggr.
	Channel [18]	Aggr.
Coordin.	Book Seller [61]	Chor.
	Buyer-Seller-Shipper [37]	Chor.
	Fan-in [6]	Chor.
	Fan-out [6]	Chor.
	GMW [6]	Chor.
	Main-Worker [65]	Multi.
	Token Ring [65]	Multi.
	Remote Function [37]	Chor.
Security	Hello Roles [37]	Chor.
	Diffie-Hellman [37]	Chor.
	Distributed Auth. [37]	Chor.
	DPrio [6]	Chor.
Games	Ticket Trader [15]*	Multi.
	Card Game [6]	Chor.
	Lottery [6]	Chor.
	Tic-Tac-Toe [6]	Chor.
Data / Serv.	Guess a Number* [22]	Multi.
	Key-value Store [6, 61]	Chor.
	Shopping List [57]	Multi.
	Graffiti RPC [57]	Multi.
	Graffiti WebSockets [57]	Multi.
	Tweet Processing [65]	Multi.
Algo. / Comp.	Consume Items [37]	Chor.
	Karatsuba [37]	Chor.
	Merge Sort [6]	Chor.
	Quicksort [37]	Chor.
	Collection [18]	Aggr.
	Collection IDs [18]	Aggr.
	Count Neighbours [18]	Aggr.
	Gradient Explicit Fields [18]	Aggr.
Spatial	Max ID [18]	Aggr.
	Neighbor Counter [18]	Aggr.
Spatial	Neighbor Distances [18]	Aggr.
	Neighbor Dist. Obstacle [18]	Aggr.

*Code from the paper since no artifact is available.

Table 2. Case Studies and Functionalities Implemented in Each Paradigm.

Case study	Multitier	Choreographic	Collective
CityEvt (3)	User registration and recommendation delivery across smartphones, edge, and cloud.	Recommendation protocol between edge nodes and the cloud.	Crowding-level monitoring from nearby smartphones.
SmrtMob (6.2.1)	Vehicle registration, credential distribution, and edge-side traffic-light coordination.	Route recommendation protocol across vehicles, edge nodes, and the cloud.	Traffic sensing from nearby vehicles to derive local traffic probes.
Nebula (6.2.2)	Dashboard reporting of learning progress and network status.	Round coordination for training, aggregation, and lifecycle management.	Distance-based neighbor selection for topology-aware participation.

which different styles can be expressed. Taken together, these implementations provide evidence of the language’s practical expressiveness.

6.1.2 Safety Guarantees. The evaluation shows that Party’s type system is effective in providing safety guarantees 3.6 for multitier, choreographic, and aggregate programming. In the RQ 1 use cases, these guarantees are assessed for each paradigm in isolation, showing that Party can statically reject unsupported or inconsistent uses of paradigm-specific constructs within each individual setting. This reduces the risk of errors and provides earlier feedback during development, without relying solely on programmer discipline. At the same time, these guarantees are obtained within a single unified framework, rather than through separate languages or type systems. Overall, this indicates that Party can improve the reliability of distributed applications while preserving expressiveness.

6.2 Inter-Paradigm Integration

To answer RQ 2, we designed case studies that combine multitier, choreographic, and aggregate programming in the same system (Table 2). For each case study, we implemented a unified solution in Party, based on explicit interactions across paradigms and their associated capabilities.

Full implementations of the case studies are provided in the companion artifact as well as the implementation of CityEvt from Section 2, which also combines the three paradigms. CityEvt spans three peer families (phones, edge nodes, and cloud), three distributed subsystems (registration, crowd monitoring, and recommendations), and two explicit cross-paradigm accesses in the recommendation workflow.

6.2.1 SmrtMob Scenario. In SmrtMob, vehicles sense roadway conditions while moving through the city, edge nodes summarize local congestion and coordinate traffic lights, and the cloud computes route recommendations from a broader traffic snapshot. The system combines on-road sensing, intersection-level coordination, and cloud-side routing decisions. The Smart Mobility case study spans three peer families (vehicles, edge nodes, and cloud), four application phases, and five explicit cross-paradigm accesses. It is implemented in ~ 500 LoC of Party code.

The phases are as follows. The registration phase uses **Multitier** to register vehicles and distribute credentials, trafficSensing uses **Collective** on vehicles to derive local traffic probes from nearby observations, signalCoordination uses **Multitier** to aggregate these probes at the edge and compute traffic-light plans, and routeRecommendation uses **Choreography** to collect route requests, send them together with the current traffic snapshot to the cloud, and multicast recommendations back to vehicles. The case study combines collective sensing on vehicles, infrastructure coordination on the edge and cloud, and a multi-party routing protocol within one program through explicit, typed placed-value handoffs.

6.2.2 Nebula federated learning system. Nebula [8, 9, 47, 48] is an open source platform for decentralized federated learning with a public codebase of about 132 K LoC that, as of March 2026, reports 76 GitHub stars. It includes a frontend (dashboard) to monitor the learning process, a controller

(orchestrator) that coordinates operation management, and a core (federated learning engine) that runs on distributed nodes. We reimplemented Nebula in Party, focusing on round coordination, topology-based participation, and dashboard reporting. We mapped the distributed locations for these components and their architectural relations to Party:

```
1 type Coordinator<: { type Tie<: Multiple[Participant] & Single[Dashboard] }
2 type Participant<: { type Tie<: Multiple[Participant] & Single[Dashboard] & Single[Coordinator] }
3 type Dashboard<: { type Tie<: Multiple[Participant] & Single[Coordinator] }
```

The *round coordination engine* orchestrates the startup, the rounds, aggregation and lifecycle of the learning process, implemented as a callback- and lock-driven runtime engine, which we implement with the **Choreography** capability, modeling the communication between the **Coordinator** and the distributed **Participant** peers as a choreographic protocol. Nebula provides different *topology-based neighbor policies* based on neighborhood distance for model aggregation. We implemented the **DistanceNeighborPolicy** as an aggregate program, leveraging the **Collective** capability to compute neighbor distances and determine which nodes should participate in the aggregation process. The *dashboard reporting* component of Nebula provides real-time monitoring of the learning process, which we implemented in Party using the **Multitier** capability, allowing the **Dashboard** to receive updates on the learning progress and display them to users.

Party’s port of Nebula combines the different paradigms safely through shared placed values with type-checked cross-paradigm accesses. The aggregate layer computes topology information, consumed by the choreographic training protocol to decide participation in each round and by the multitier dashboard to report the current status. The choreographic layer produces the global model, round summaries, and the recent local updates, consumed by the multitier dashboard.

The round lifecycle in the original implementation spreads across 22 message callbacks, which are expressed in Party as a choreographic communication protocol with no callbacks – without inversion of control flow. We further replaced 57 direct inter-dependencies between the policy code and the communication infrastructure in the topology stack with a single aggregate program, abstracting away the details of neighbor discovery and communication and ensuring resilience to network changes. Finally, the original networking layer needs 60 different kinds of messages and actions to model communication across hosts, which the Party version replaces with seven explicit cross-host communication terms, combining expressions from different hosts in a multitier fashion.

6.3 Portability

For **RQ 3**, we assess whether capabilities can serve as a general unifying abstraction for multitier, choreographic, and aggregate programming, independent of a specific capability mechanism or host language beyond Scala. To this end, we develop prototype implementations of the design underlying Party in other languages and capability systems.

6.3.1 Koka. We model Party’s capabilities with the algebraic effects of the Koka functional programming language [43]. Placement types are represented as an algebraic data type, and the ability to place values is modeled as an effectful operation:

```
1 type on<v, p>
2 effect fun take(pv: on<v, p>): v
3 effect placementtype
4 fun on(peer: p, body: () -> <take> v): on<v, p>
```

We define an **on** type to model placement types while the **placementtype** effect defines the **on** operation, taking a peer type and an effectful body and returning a placed value of type **on<v, p>**. The **take** effect allows access to a locally placed value. Notably, the **on** body is effectful, indicating

that it may involve placed value access, creating a similar capability-based structure as in Party. As the Koka type system does not support bounds on generic type parameters, we introduce explicit witness parameters to represent tie relationships:

```
1 abstract type peer<p>
2 struct tiesingle<p1, p2> { peer1: peer<p1>; peer2: peer<p2> }
3 struct tiemultiple<p1, p2> { peer1: peer<p1>; peer2: peer<p2> }
```

We define abstract peer types and concrete structures to represent single and multiple ties. We then add additional parameters to accept the corresponding tie witness to the effects that need to enforce tie relationships (e.g., communication). For example, the choreo effect defines communication operations that require tie witnesses to ensure valid connections between peers:

```
1 effect choreo
2 fun comm(from: f, to: t, tie: tiesingle<f,t>, pv: on<v,f>): on<v,t>
3 fun broadcast(from: f, to: t, tie: tiemultiple<f,t>, pv: on<v,f>): on<v,t>
```

Koka's algebraic effect system is similar to Scala 3's capture checking: effects are explicitly tracked in function types and effect handlers delimit the scope where operations are available, preventing capabilities from escaping the intended context. As a result, the *safe boundary* and *leak prevention* properties (Section 3.6) can be enforced in Koka through its native effect typing discipline. Since Koka does not support separation checking – which in Party's Scala 3 implementation prevents capability aliasing – there is no mechanism for *deadlock avoidance* (Section 3.6).

6.3.2 Rust. The lack of native support for contextual abstractions in Rust makes it challenging to directly model capabilities as first-class entities. Similar to *Chorus* [6], we use traits to represent types of capabilities, and capabilities are explicitly passed to functions requiring them. The borrowing mechanism prevents leaking placed values across peer boundaries (see Section 3.6).

```
1 type L = LocationSet!(Alice, Bob);
2 fn run(self, op: &impl ChoreoOp<Self::L>) {
3   let value_on_alice = op.locally(Alice, |un| { "Hello from Alice!".to_string() });
4   let fun_at_alice = op.locally(Alice, |un| { |un| { un.unwrap(&value_on_alice).clone() } });
5   let msg_at_bob = op.comm(Alice, Bob, &fun_at_alice);
6   op.locally(Bob, |un| {
7     let f = un.unwrap(&msg_at_bob);
8     f();
9   });
10 }
```

In the example, we place a closure on *Alice*, which captures a string on *Alice* (Line 4), and we transmit the closure to *Bob*. This program is not allowed since *Bob* could invoke the closure which accesses to the value on *Alice* but there is no explicit communication of the captured value to *Bob*. The Rust borrow checker prevents this code from compiling, since the closure would outlive the placed value it captures, violating Rust's borrowing rules.

In another example, we introduce a nested scope (Line 4) where *Alice* waits for a message from *Bob* (Line 5), while *Bob* simultaneously waits for a message from *Alice* (Line 9):

```
1 type L = LocationSet!(Alice, Bob);
2 fn run(self, op: &impl ChoreoOp<Self::L>) {
3   let msg_at_alice = op.locally(Alice, |un| {
4     let on_bob = op.locally(Bob, |un| { "Hello from Bob!".to_string() });
5     let on_alice = op.comm(Bob, Alice, &on_bob);
6     let msg = un.unwrap(&on_alice);
```

```

1030 7   msg.clone()
1031 8   });
1032 9   let msg_at_bob = op.comm(Alice, Bob, &msg_at_alice);
1033 10  op.locally(Bob, |un| { un.unwrap(&msg_at_bob) });
1034 11 }

```

The code above leads to a deadlock which ChoRus’s API cannot prevent at compile time. Yet, this is a design choice of ChoRus rather than a fundamental limitation of Rust: the choreography operator is exposed to user code as a shared resource, so the type system does not enforce that at most one placement operation is active at a time. In principle, treating the operator as an exclusive resource would enable Rust’s type system to statically reject nested uses, yielding similar guarantees to those of our approach. In Party, we leverage *separation checking* on the **Placement** capability to statically detect and reject such nesting (Section 4.1).

Limited Support for Nesting. While Section 6.1 shows that Party can express choreographic, multitier, and aggregate programming, and Section 6.2 shows how these paradigms can be combined in Party, the issue of nesting placed scopes discussed in Section 6.3 is the main limitation of Party compared to existing multitier languages, which typically allow nesting placement scopes. As nesting can lead to subtle deadlock conditions if not carefully managed, our current design rejects nesting statically. Appendix A elaborates further on this issue and our solution.

7 Related Work

Multitier Programming. At their core, multitier languages allow developers to define the tiers to which different pieces of the code belong. The language features that enable this definition are quite diverse across multitier languages – ranging from staging to annotations to types.

Hop [59] is a dynamically typed Scheme-based language using asynchronous callbacks for received messages and return values. A recent line of work has ported Hop to a JavaScript-based framework, Hop.js [60]. Links [23] is a statically typed language where functions are annotated with either client- or server-side execution. Links translates to SQL for the database tier and to JavaScript for the web browser. Recent work extended Links with algebraic effects [40], provenance tracking [34], session types [44] exception handling [36], language-integrated queries for heterogeneous nested collections [58], temporal data [35] and place polymorphism [21]. ScalaLoc [65] specifically targets generic distributed systems, i.e., not only client–server. Placement types assign locations to data and computations. ScalaLoc has been extended with place polymorphism for reusable (parametrized) multitier modules [66] and with dynamic placement using union types [69]. Weisenburger et al. [67] survey the state of the art in multitier programming.

Choreographic Programming. In choreographic programming, a distributed program is defined as a global description of the interactions of a distributed system’s components [42, 50, 64]. Similar to multitier programming, the compiler automatically produces a correct implementation for each component [14]. Choreographic programming’s formal foundations are rooted in process calculi [5]. It has been used to investigate new techniques on information flow control [45], deadlock-free distributed algorithms [27] and protocols for dynamic run time code updates for components [55]. Pirouette [41] is implemented in Coq [24] for a mechanized proof for deadlock freedom. Role parameters in the choreographic language Choral [37] enable parametric place polymorphism. Chorλ [26] supports a restricted kind of place polymorphism, where only top-level definitions can be defined to be place-polymorphic. PolyChorλ [39] extends Chorλ with complete support place polymorphism. ChoRus [6] introduces features, such as enclaves, multiply-located values, and census polymorphism, to improve the efficiency and portability of library-based choreographic programming across host languages.

Aggregate Programming. Aggregate programming is a programming paradigm for collective adaptive systems, in which a single global program specifies the behavior of an entire ensemble of situated devices (sensors, robots, smartphones, edge nodes) [63]. The foundation is the field calculus [2, 4], where a program computes computational fields: mappings from devices to values evolving over space and time under asynchronous sense-compute-act rounds [19, 63]. This model supports compositional construction of self-organizing algorithms for tasks such as information diffusion, gradients, and clustering, with provable properties like self-stabilization.

Recent work lets developers write high-level field transformations, while the runtime is responsible for distributed coordination and fault tolerance [16, 17]. Further research explored how to leverage not only end devices for aggregate computations, but also edge nodes or the cloud [31–33]; while other work investigated how to integrate AI techniques into aggregate programming [1, 28].

The three main implementations are Protelis [52], a Java-based language, Scafi [18], an internal Scala DSL to express field calculus-style programs, and FCPP [3], an optimized C++ implementation.

Capabilities and Effect Systems. Our work builds on a long line of research on effect systems and capability-based programming. Effect systems extend type systems with information about computational effects, enabling static reasoning about side effects such as state mutation, exceptions, I/O, or concurrency [46]. More recently, algebraic effects and handlers have emerged as a structured and compositional approach to effectful programming, decoupling effect declaration from interpretation and supporting modular reasoning about side effects [53, 54]. Languages such as *Effekt* demonstrate how algebraic effects can be integrated with capability-based reasoning to provide fine-grained control over effect scoping and effect safety, combining effect polymorphism with region-based reasoning to prevent unintended capability escape [12].

Scala 3 adopts a related approach through contextual abstractions and capture checking, models capabilities as implicit context parameters and tracks them through types [30]. Here, capabilities are values that grant access to specific operations, and the type system ensures that they cannot escape their scope. This approach connects to classical capability-based security models, where possession of a capability determines authority [49], and to recent work on lightweight effect tracking via contextual abstractions [10, 11, 68]. In our design, we use such capabilities to constrain multiparty programming abstractions while their interaction is regulated via placement types.

8 Conclusion

In this paper, we propose capabilities as a unifying abstraction to model multitier programming, choreographic programming, and aggregate programming. Our approach enables the safe composition of such paradigms in the same system while preserving their individual abstractions and guarantees. The design, formalization and implementation demonstrate that this unifying perspective is not specific to a particular language or capability mechanism, but applies more generally. We believe that modeling multiparty programming through capabilities provides a principled foundation for the design of heterogeneous distributed systems.

9 Data-Availability Statement

The artifact for this paper has been uploaded in anonymized form as supplementary material with the initial submission. The artifact contains the Scala 3 reference implementation of Party, the examples discussed in the paper, the use cases, and the case studies for the evaluation. While all the code developed to support this work is available in the artifact, the artifact, in its current state, should be regarded as a snapshot of the author's code rather than a polished archival release, and it may require a matching toolchain and some manual setup. There is no separate dataset associated with this work. We intend to submit a polished version for Artifact Evaluation.

References

- [1] Gianluca Aguzzi, Roberto Casadei, and Mirko Viroli. 2022. Towards reinforcement learning-based aggregate computing. In *International Conference on Coordination Languages and Models*. Springer, 72–91.
- [2] Giorgio Audrito, Roberto Casadei, Ferruccio Damiani, Guido Salvaneschi, and Mirko Viroli. 2024. The eXchange Calculus (XC): A functional programming language design for distributed collective systems. *J. Syst. Softw.* 210 (2024), 111976. doi:10.1016/J.JSS.2024.111976
- [3] Giorgio Audrito and Gianluca Torta. 2024. FCPP to aggregate them all. *Sci. Comput. Program.* 231 (2024), 103026. doi:10.1016/J.SCICO.2023.103026
- [4] Giorgio Audrito, Mirko Viroli, Ferruccio Damiani, Danilo Pianini, and Jacob Beal. 2019. A Higher-Order Calculus of Computational Fields. *ACM Transactions on Computational Logic* 20, 1, Article 5 (Jan. 2019), 55 pages. doi:10.1145/3285956
- [5] J. C. M. Baeten. 2005. A Brief History of Process Algebra. *Theor. Comput. Sci.* 335, 2–3 (may 2005), 131–146. doi:10.1016/j.tcs.2004.07.036
- [6] Mako Bates, Shun Kashiwa, Syed Jafri, Gan Shen, Lindsey Kuper, and Joseph P. Near. 2025. Efficient, Portable, Census-Polymorphic Choreographic Programming. *Proc. ACM Program. Lang.* 9, PLDI (2025), 1143–1166. doi:10.1145/3729296
- [7] Jacob Beal, Danilo Pianini, and Mirko Viroli. 2015. Aggregate Programming for the Internet of Things. *Computer* 48, 9 (2015), 22–30. doi:10.1109/MC.2015.261
- [8] Enrique Tomás Martínez Beltrán, Mario Quiles Pérez, Pedro Miguel Sánchez Sánchez, Sergio López Bernal, Jérôme Bovet, Manuel Gil Pérez, Gregorio Martínez Pérez, and Alberto Huertas Celdrán. 2023. Decentralized federated learning: Fundamentals, state of the art, frameworks, trends, and challenges. *IEEE Communications Surveys & Tutorials* 25, 4 (2023), 2983–3013.
- [9] Martínez Beltrán. 2024. NEBULA: A Platform for Decentralized Federated Learning. <https://github.com/CyberDataLab/nebula>.
- [10] Aleksander Boruch-Gruszecki, Martin Odersky, Edward Lee, Ondrej Lhoták, and Jonathan Immanuel Brachthäuser. 2023. Capturing Types. *ACM Trans. Program. Lang. Syst.* 45, 4 (2023), 21:1–21:52. doi:10.1145/3618003
- [11] Jonathan Immanuel Brachthäuser, Philipp Schuster, and Klaus Ostermann. 2020. Effects as capabilities: effect handlers and lightweight effect polymorphism. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 126 (Nov. 2020), 30 pages. doi:10.1145/3428194
- [12] Jonathan Immanuel Brachthäuser, Philipp Schuster, and Klaus Ostermann. 2021. Effekt: Capability-Passing Style for Effect Safety. In *Proceedings of the 35th European Conference on Object-Oriented Programming (ECOOP)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [13] Marco Carbone, Kohei Honda, and Nobuko Yoshida. 2012. Structured Communication-Centered Programming for Web Services. *ACM Trans. Program. Lang. Syst.* 34, 2, Article 8 (June 2012), 78 pages. doi:10.1145/2220365.2220367
- [14] Marco Carbone and Fabrizio Montesi. 2013. Deadlock-Freedom-by-Design: Multiparty Asynchronous Global Programming. In *Proceedings of the 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Rome, Italy) (POPL '13). ACM, New York, NY, USA, 263–274. doi:10.1145/2429069.2429101
- [15] Andoni Lombide Carreton, Stijn Mostinckx, Tom Van Cutsem, and Wolfgang De Meuter. 2010. Loosely-Coupled Distributed Reactive Programming in Mobile Ad Hoc Networks. In *Objects, Models, Components, Patterns, 48th International Conference, TOOLS 2010, Málaga, Spain, June 28 - July 2, 2010. Proceedings (Lecture Notes in Computer Science, Vol. 6141)*, Jan Vitek (Ed.). Springer, 41–60. doi:10.1007/978-3-642-13953-6_3
- [16] Roberto Casadei. 2023. Macroprogramming: Concepts, State of the Art, and Opportunities of Macroscopic Behaviour Modelling. *ACM Comput. Surv.* 55, 13s (2023), 275:1–275:37. doi:10.1145/3579353
- [17] Roberto Casadei and Mirko Viroli. 2024. Declarative Macro-Programming of Collective Systems with Aggregate Computing: An Experience Report. In *Proceedings of the 26th International Symposium on Principles and Practice of Declarative Programming* (Milano, Italy) (PPDP '24). Association for Computing Machinery, New York, NY, USA, Article 5, 5 pages. doi:10.1145/3678232.3678235
- [18] Roberto Casadei, Mirko Viroli, Gianluca Aguzzi, and Danilo Pianini. 2022. Scaff: A scala DSL and toolkit for aggregate programming. *SoftwareX* 20 (2022), 101248.
- [19] Roberto Casadei, Mirko Viroli, Giorgio Audrito, Danilo Pianini, and Ferruccio Damiani. 2021. Engineering collective intelligence at the edge with aggregate processes. *Engineering Applications of Artificial Intelligence* 97 (2021), 104081. doi:10.1016/j.engappai.2020.104081
- [20] Elias Castegren and Tobias Wrigstad. 2016. Reference Capabilities for Concurrency Control. In *30th European Conference on Object-Oriented Programming (ECOOP 2016) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 56)*, Shriram Krishnamurthi and Benjamin S. Lerner (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 5:1–5:26. doi:10.4230/LIPIcs.ECOOP.2016.5
- [21] Kwanghoon Choi, James Cheney, Simon Fowler, and Sam Lindley. 2020. A Polymorphic RPC Calculus. *Science of Computer Programming* 197 (2020), 102499. doi:10.1016/j.scico.2020.102499

- [22] Stephen Chong, Jed Liu, Andrew C. Myers, Xin Qi, K. Vikram, Lantian Zheng, and Xin Zheng. 2007. Secure web applications via automatic partitioning. In *Proceedings of Twenty-First ACM SIGOPS Symposium on Operating Systems Principles* (Stevenson, Washington, USA) (SOSP '07). ACM, New York, NY, USA, 31–44. doi:10.1145/1294261.1294265
- [23] Ezra Cooper, Sam Lindley, Philip Wadler, and Jeremy Yallop. 2006. Links: Web Programming without Tiers. In *Proceedings of the 5th International Conference on Formal Methods for Components and Objects* (Amsterdam, The Netherlands) (FMCO'06). Springer-Verlag, Berlin, Heidelberg, 266–296.
- [24] Coq Development Team. 2016. *The Coq Proof Assistant, version 8.11.0*. <http://coq.inria.fr/>
- [25] Karl Cray, David Walker, and Greg Morrisett. 1999. Typed memory management in a calculus of capabilities. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Antonio, Texas, USA) (POPL '99). Association for Computing Machinery, New York, NY, USA, 262–275. doi:10.1145/292540.292564
- [26] Luís Cruz-Filipe, Eva Graversen, Lovro Lugović, Fabrizio Montesi, and Marco Peressotti. 2022. Functional Choreographic Programming. In *Theoretical Aspects of Computing – ICTAC 2022*, Helmut Seidl, Zhiming Liu, and Corina S. Pasareanu (Eds.). Springer International Publishing, Cham, 212–237.
- [27] Luís Cruz-Filipe and Fabrizio Montesi. 2016. Choreographies in Practice. In *Proceedings of the 36th IFIP International Conference on Formal Techniques for Distributed Objects, Components, and Systems* (Heraklion, Greece) (FORTE '16), Elvira Albert and Ivan Lanese (Eds.). Springer-Verlag, Berlin, Heidelberg, 114–123. doi:10.1007/978-3-319-39570-8_8
- [28] Davide Domini, Nicolas Farabegoli, Gianluca Aguzzi, Mirko Viroli, and Lukas Esterle. 2026. Decentralized proximity-aware clustering for collective self-federated learning. *Internet Things* 35 (2026), 101841. doi:10.1016/J.IOT.2025.101841
- [29] Anton Eklblad and Koen Claessen. 2014. A seamless, client-centric programming model for type safe web applications. In *Proceedings of the 2014 ACM SIGPLAN symposium on Haskell, Gothenburg, Sweden, September 4-5, 2014*, Wouter Swierstra (Ed.). ACM, 79–89. doi:10.1145/2633357.2633367
- [30] Martin Odersky et al. 2021. Scala 3 Reference. <https://docs.scala-lang.org/scala3/reference/>.
- [31] Nicolas Farabegoli, Danilo Pianini, Roberto Casadei, and Mirko Viroli. 2024. Dynamic IoT deployment reconfiguration: A global-level self-organisation approach. *Internet Things* 28 (2024), 101412. doi:10.1016/J.IOT.2024.101412
- [32] Nicolas Farabegoli, Danilo Pianini, Roberto Casadei, and Mirko Viroli. 2024. Scalability through Pulverisation: Declarative deployment reconfiguration at runtime. *Future Gener. Comput. Syst.* 161 (2024), 545–558. doi:10.1016/J.FUTURE.2024.07.042
- [33] Nicolas Farabegoli, Mirko Viroli, and Roberto Casadei. 2024. Flexible Self-organisation for the Cloud-Edge Continuum: a Macro-programming Approach. In *IEEE International Conference on Autonomic Computing and Self-Organizing Systems, ACSOS 2024, Aarhus, Denmark, September 16-20, 2024*. IEEE, 21–30. doi:10.1109/ACSOS61780.2024.00020
- [34] Stefan Fehrenbach and James Cheney. 2019. Language-integrated provenance by trace analysis. In *Proceedings of the 17th ACM SIGPLAN International Symposium on Database Programming Languages* (Phoenix, AZ, USA) (DBPL 2019). ACM, New York, NY, USA, 74–84. doi:10.1145/3315507.3330198
- [35] Simon Fowler, Vashti Galpin, and James Cheney. 2022. Language-Integrated Query for Temporal Data. In *Proceedings of the 21st ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences* (Auckland, New Zealand) (GPCE 2022). ACM, New York, NY, USA, 5–19. doi:10.1145/3564719.3568690
- [36] Simon Fowler, Sam Lindley, J. Garrett Morris, and Sára Decova. 2019. Exceptional asynchronous session types: session types without tiers. *Proc. ACM Program. Lang.* 3, POPL, Article 28 (jan 2019), 29 pages. doi:10.1145/3290341
- [37] Saverio Giallorenzo, Fabrizio Montesi, and Marco Peressotti. 2024. Choral: Object-oriented Choreographic Programming. *ACM Trans. Program. Lang. Syst.* 46, 1, Article 1 (jan 2024), 59 pages. doi:10.1145/3632398
- [38] Saverio Giallorenzo, Fabrizio Montesi, Marco Peressotti, David Richter, Guido Salvaneschi, and Pascal Weisenburger. 2021. Multiparty Languages: The Choreographic and Multitier Cases (Pearl). In *35th European Conference on Object-Oriented Programming, ECOOP 2021, July 11-17, 2021, Aarhus, Denmark (Virtual Conference)* (LIPICs, Vol. 194), Anders Møller and Manu Sridharan (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 22:1–22:27. doi:10.4230/LIPICs.ECOOP.2021.22
- [39] Eva Graversen, Andrew K. Hirsch, and Fabrizio Montesi. 2024. Alice or Bob?: Process polymorphism in choreographies. *Journal of Functional Programming* 34 (2024). doi:10.1017/S0956796823000114
- [40] Daniel Hillerström, Sam Lindley, Robert Atkey, and KC Sivaramakrishnan. 2017. Continuation Passing Style for Effect Handlers. In *Proceedings of the 2nd International Conference on Formal Structures for Computation and Deduction (FSCD '17)* (Oxford, UK) (*Leibniz International Proceedings in Informatics (LIPIcs)*, Vol. 84), Dale Miller (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, Article 18, 19 pages. doi:10.4230/LIPICs.FSCD.2017.18
- [41] Andrew K. Hirsch and Deepak Garg. 2022. Pirouette: Higher-Order Typed Functional Choreographies. *Proc. ACM Program. Lang.* 6, POPL, Article 23 (jan 2022), 27 pages. doi:10.1145/3498684
- [42] Ivan Lanese, Claudio Guidi, Fabrizio Montesi, and Gianluigi Zavattaro. 2008. Bridging the Gap between Interaction- and Process-Oriented Choreographies. In *Proceedings of the 2008 Sixth IEEE International Conference on Software Engineering and Formal Methods (SEFM '08)*. IEEE Computer Society, USA, 323–332. doi:10.1109/SEFM.2008.11

- [43] Daan Leijen. 2014. Koka: Programming with Row Polymorphic Effect Types. In *Proceedings 5th Workshop on Mathematically Structured Functional Programming, MSFP@ETAPS 2014, Grenoble, France, 12 April 2014 (EPTCS, Vol. 153)*, Paul Blain Levy and Neel Krishnaswami (Eds.). 100–126. doi:10.4204/EPTCS.153.8
- [44] Sam Lindley and J. Garrett Morris. 2017. Lightweight Functional Session Types. In *Behavioural Types: From Theory to Tools*, Simon Gay and António Ravara (Eds.). River Publishers, Chapter 12. doi:10.13052/rp-9788793519817
- [45] Alberto Lluch Lafuente, Flemming Nielson, and Hanne Riis Nielson. 2015. *Discretionary Information Flow Control for Interaction-Oriented Specifications*. Lecture Notes in Computer Science, Vol. 9200. Springer-Verlag, Berlin, Heidelberg, 427–450. doi:10.1007/978-3-319-23165-5_20
- [46] John M. Lucassen and David K. Gifford. 1988. Polymorphic Effect Systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 47–57.
- [47] Enrique Tomás Martínez Beltrán, Ángel Luis Perales Gómez, Chao Feng, Pedro Miguel Sánchez Sánchez, Sergio López Bernal, Gérôme Bovet, Manuel Gil Pérez, Gregorio Martínez Pérez, and Alberto Huertas Celdrán. 2024. Fedstellar: A Platform for Decentralized Federated Learning. *Expert Systems with Applications* 242 (2024), 122861. doi:10.1016/j.eswa.2023.122861
- [48] Enrique Tomás Martínez Beltrán, Pedro Miguel Sánchez Sánchez, Sergio López Bernal, Gérôme Bovet, Manuel Gil Pérez, Gregorio Martínez Pérez, and Alberto Huertas Celdrán. 2023. Fedstellar: A Platform for Training Models in a Privacy-preserving and Decentralized Fashion. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI-23*, Edith Elkind (Ed.). International Joint Conferences on Artificial Intelligence Organization, 7154–7157. doi:10.24963/ijcai.2023/838 Demo Track.
- [49] Mark S. Miller. 2006. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. Johns Hopkins University. PhD Thesis.
- [50] Fabrizio Montesi. 2014. Kickstarting Choreographic Programming. In *Proceedings of the 13th International Workshop on Web Services and Formal Methods (Eindhoven, Netherlands) (WS-FM '14)*, Thomas Hildebrandt, António Ravara, Jan Martijn van der Werf, and Matthias Weidlich (Eds.). Springer-Verlag, Berlin, Heidelberg, 3–10. doi:10.1007/978-3-319-33612-1_1
- [51] Fabrizio Montesi. 2023. *Introduction to Choreographies*. Cambridge University Press.
- [52] Danilo Pianini, Mirko Viroli, and Jacob Beal. 2015. Protelis: practical aggregate programming. In *Proceedings of the 30th Annual ACM Symposium on Applied Computing*. 1846–1853.
- [53] Gordon D. Plotkin and John Power. 2003. Algebraic Operations and Generic Effects. *Applied Categorical Structures* 11, 1 (2003), 69–94.
- [54] Gordon D. Plotkin and Matija Pretnar. 2009. Handlers of Algebraic Effects. In *Proceedings of the 18th European Symposium on Programming (ESOP)*. Springer, 80–94.
- [55] Mila Dalla Preda, Maurizio Gabbriellini, Saverio Giallorenzo, Ivan Lanese, and Jacopo Mauro. 2017. Dynamic Choreographies: Theory And Implementation. *Logical Methods in Computer Science* 13, 2 (April 2017), 57 pages. doi:10.23638/LMCS-13(2:1)2017
- [56] Bob Reynders, Dominique Devriese, and Frank Piessens. 2014. Multi-Tier Functional Reactive Programming for the Web. In *Onward! 2014, Proceedings of the 2014 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software, part of SPLASH '14, Portland, OR, USA, October 20-24, 2014*, Andrew P. Black, Shriram Krishnamurthi, Bernd Bruegge, and Joseph N. Ruskiewicz (Eds.). ACM, 55–68. doi:10.1145/2661136.2661140
- [57] Bob Reynders, Michael Greefs, Dominique Devriese, and Frank Piessens. 2018. Scalagna 0.1: towards multi-tier programming with Scala and Scala.js. In *Conference Companion of the 2nd International Conference on Art, Science, and Engineering of Programming, Nice, France, April 09-12, 2018*, Stefan Marr and Jennifer B. Sartor (Eds.). ACM, 69–74. doi:10.1145/3191697.3191731
- [58] Wilmer Ricciotti and James Cheney. 2021. Query Lifting: Language-integrated query for heterogeneous nested collections. In *Programming Languages and Systems (Lecture Notes in Computer Science, Vol. 12648)*, Nobuko Yoshida (Ed.). Springer International Publishing, Cham, 579–606. doi:10.1007/978-3-030-72019-3_21
- [59] Manuel Serrano, Erick Galesio, and Florian Loitsch. 2006. Hop, A Language for Programming the Web 2.0. In *Companion to the 21th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (Portland, OR, USA) (OOPSLA Companion '06)*. ACM, New York, NY, USA.
- [60] Manuel Serrano and Vincent Prunet. 2016. A glimpse of Hopjs. In *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming (Nara, Japan) (ICFP 2016)*. ACM, New York, NY, USA, 180–192. doi:10.1145/2951913.2951916
- [61] Gan Shen, Shun Kashiwa, and Lindsey Kuper. 2023. HasChor: Functional Choreographic Programming for All (Functional Pearl). *Proc. ACM Program. Lang.* 7, ICFP, Article 207 (aug 2023), 25 pages. doi:10.1145/3607849
- [62] Mirko Viroli, Giorgio Audrito, Jacob Beal, Ferruccio Damiani, and Danilo Pianini. 2018. Engineering Resilient Collective Adaptive Systems by Self-Stabilisation. *ACM Trans. Model. Comput. Simul.* 28, 2 (2018), 16:1–16:28. doi:10.1145/3177774

- [63] Mirko Viroli, Jacob Beal, Ferruccio Damiani, Giorgio Audrito, Roberto Casadei, and Danilo Pianini. 2019. From distributed coordination to field calculus and aggregate computing. *Journal of Logical and Algebraic Methods in Programming* 109 (2019), 100486. doi:10.1016/j.jlamp.2019.100486
- [64] W3C WS-CDL Working Group. 2005. *Web Services Choreography Description Language Version 1.0*. <http://www.w3.org/TR/ws-cdl-10/>
- [65] Pascal Weisenburger, Mirko Köhler, and Guido Salvaneschi. 2018. Distributed system development with ScalaLoc. *Proc. ACM Program. Lang.* 2, OOPSLA (2018), 129:1–129:30. doi:10.1145/3276499
- [66] Pascal Weisenburger and Guido Salvaneschi. 2019. Multitier Modules. In *Proceedings of the 33rd European Conference on Object-Oriented Programming (ECOOP '19)* (London, UK) (*Leibniz International Proceedings in Informatics (LIPIcs)*, Vol. 134), Alastair F. Donaldson (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, Article 3, 29 pages. doi:10.4230/LIPIcs.ECOOP.2019.3
- [67] Pascal Weisenburger, Johannes Wirth, and Guido Salvaneschi. 2021. A Survey of Multitier Programming. *ACM Comput. Surv.* 53, 4 (2021), 81:1–81:35. doi:10.1145/3397495
- [68] Yichen Xu, Aleksander Boruch-Gruszecki, and Martin Odersky. 2024. Degrees of Separation: A Flexible Type System for Safe Concurrency. *Proc. ACM Program. Lang.* 8, OOPSLA1 (2024), 1181–1207. doi:10.1145/3649853
- [69] George Zakhour, Pascal Weisenburger, and Guido Salvaneschi. 2023. Type-Safe Dynamic Placement with First-Class Placed Values. *Proceedings of the ACM on Programming Languages* 7, OOPSLA2, Article 297 (Oct. 2023), 29 pages. doi:10.1145/3622873

A Comparison with ChoRus

ChoRus [6] is a Rust library for choreographic programming, which supports enclaves (i.e., a restricted section of the choreography involving a subset of participants), and differentiated communication primitives (e.g., broadcast, multicast) other than the traditional point-to-point interaction. In this section, we compare Party with *ChoRus*, focusing on how Party's type system prevents deadlocks at compile time, and similar safety features the two systems share.

A.1 Prevent Deadlock

Both Party and *ChoRus* share similar safety features, we identified a programming pattern in *ChoRus* that leads to a deadlock at runtime, without intercepting such scenario at compile time. Consider the following choreography in *ChoRus*:

Listing 1. Deadlocking choreography in *ChoRus*.

```
1 type L = LocationSet!(Alice, Bob);
2 fn run(self, op: &impl ChoreoOp<Self::L>) {
3     let msg_at_alice = op.locally(Alice, |un| {
4         let on_bob = op.locally(Bob, |_| { "Hello from Bob!".to_string() });
5         let on_alice = op.comm(Bob, Alice, &on_bob);
6         let msg = un.unwrap(&on_alice);
7         msg.clone()
8     });
9     let msg_at_bob = op.comm(Alice, Bob, &msg_at_alice);
10    op.locally(Bob, |un| { un.unwrap(&msg_at_bob) });
11 }
```

In Listing 1, we mistakenly try to nest **locally** constructs for different participants, in particular we are nesting within **Alice**'s local computation a **locally** for **Bob**. Since the Endpoint Projection (EPP) is performed at runtime, this choreography leads to a deadlock, because the outer **Alice**'s block is not evaluated when the choreography is executed on **Bob**'s side, thus **Bob** never reaches its inner **locally** block, leading to a deadlock. Even though this violates the intended use of *ChoRus*, nothing prevents the programmer from writing such code, and the type system does not catch this error at compile time.

A.2 Prevent unsafe placed function calls

Consider the code snippet in Listing 2 where a string is created at **Alice**, then a function at **Alice** is created to return that string, and that function is sent to **Bob** to be executed there:

Listing 2. Unsafe placed function call in *ChoRus*.

```
1 type L = LocationSet!(Alice, Bob);
2 fn run(self, op: &impl ChoreoOp<Self::L>) {
3     let value_on_alice = op.locally(Alice, |un| {
4         "Hello from Alice!".to_string()
5     });
6     let fun_at_alice = op.locally(Alice, |un| {
7         || { un.unwrap(&value_on_alice).clone() }
8     });
9     let msg_at_bob = op.comm(Alice, Bob, &fun_at_alice);
10    op.locally(Bob, |un| {
11        let f = un.unwrap(&msg_at_bob);
12        f()
13    });
14 }
```

This code cannot be considered safe, because the function created at **Alice** closes over a value that only exists at **Alice**, and when that function is sent to **Bob** and executed there, it will try to access a value that does not exist at **Bob**, leading to an error.

In Party, we can avoid such unsafe placed function calls at compile time, by leveraging the new Scala 3 *Capture Checking* feature. Consider the similar code snippet in Party shown in [Listing 3](#).

Listing 3. Unsafe placed function call in Party.

```
1 type Alice <: { type Tie <: Single[Bob] }
2 type Bob <: { type Tie <: Single[Alice] }
3 def choreography(using Choreography, Network): Unit =
4   val valueOnAlice: String on Alice = on[Alice] { "Hello from Alice!" }
5   val funAtAlice: (() => String) on Alice = on[Alice]:
6     () => take(valueOnAlice)
7   val msgAtBob: (() => String) on Bob = comm[Alice, Bob](funAtAlice)
8   on[Bob]:
9     val f: () => String = take(msgAtBob)
10    f()
```

In [Listing 3](#), we define an equivalent choreography using Party. However, the Scala 3 capture checking feature prevents us from creating the function at **Alice** that closes over the value created at **Alice**. This is because of the same **Label** capability described in the previous section, which is tracked by the type system, and prevents any capturing of such capability that may leak outside its scope, as we are trying to do in this example. The **Label** capability is captured because of the use of the **take** extension method that requires the **Label** capability to be in scope. This has two benefits: (i) it prevents us from calling **take** outside of a local peer context, leveraging the **Label** capability, and (ii) it allows to track the capability and avoid capturing it in functions that may be sent to other peers, violating placement constraints.

B Type Soundness Proofs

THEOREM B.1 (PROGRESS). *For an architecture-satisfying peer configuration $\Omega \models \mathcal{A}$ with consistent VM states, If π_i^P is a peer instance in Ω with expression e such that $\mathcal{A}, \Gamma \vdash e : T$ with C , then either e is a value or $\Omega; \sigma \triangleright e \rightarrow^P \pi_i^P \Omega'; \sigma' \triangleright e'$ with consistent VM states.*

PROOF OF THEOREM B.1. By induction on the typing derivation of $\mathcal{A}, \Gamma \vdash e : T$ with C . We proceed by case analysis on the form of e .

- **Case $e = v$** – then e is already a value.
- **Case $e = e_1 e_2$** – by inversion of the typing rule, we have $\mathcal{A}, \Gamma \vdash e_1 : T_1 \rightarrow T$ with C_1 and $\mathcal{A}, \Gamma \vdash e_2 : T_1$ with C_2 . By the induction hypothesis, either e_1 is a value or can step. If e_1 can step, then by **E-AppE1**, e can step. If e_1 is a value, then either e_2 can step (and e steps by **E-AppE2**), or e_2 is a value and $e_1 = \lambda x : T_1. e_0$ by canonical forms, allowing e to step by **E-App**.
- **Case $e = \text{on } P \ e_1$** – by inversion, $\mathcal{A}, \Gamma \vdash e_1 : T$ with C . By induction hypothesis, either e_1 is a value (allowing **E-OnV** to apply) or e_1 can step (allowing **E-One** to apply).
- **Case $e = \text{asLocal } e_1 \text{ on } P_r$** – by well-formedness of $\Omega \models \mathcal{A}$, the required tie relationship exists. Either e_1 can step (**E-AsLocalE** applies) or e_1 is a value (**E-AsLocalV** applies).
- **Case $e = \text{take } e_1 \text{ on } P$** – by inversion, $\mathcal{A}, \Gamma \vdash e_1 \text{ on } P : T \text{ on } P$ with C . Either e_1 can step (**E-TakeE** applies) or e_1 is a value (**E-TakeV** applies).
- **Case $e = \text{comm } e_1 \text{ on } P_l$** – by inversion and well-formedness of $\Omega \models \mathcal{A}$, a valid tie relationship from P_l to some P_r exists in the architecture. Either e_1 can step (**E-CommE** applies) or e_1 is a value (**E-CommV** applies).
- **Case $e = \text{collective } e_1$** – by inversion, $\mathcal{A}, \Gamma \vdash e_1 : T$ with C . Either e_1 can step (**E-CollE** applies) or e_1 is a value (**E-CollV** applies).
- Other cases (rep, nbr, branch, run) follow similarly by induction and the existence of corresponding evaluation rules.

□

THEOREM B.2 (PRESERVATION). *For an architecture-satisfying peer configuration $\Omega \models \mathcal{A}$ with consistent VM states, if $\mathcal{A}, \Gamma \vdash e : T$ with C for some expression e on peer instance π_i^P , and $\Omega; \sigma \triangleright e \rightarrow^P \pi_i^P \Omega'; \sigma' \triangleright e'$, then $\mathcal{A}, \Gamma \vdash e' : T$ with C with consistent VM states.*

PROOF OF THEOREM B.2. By induction on the reduction rules $\Omega; \sigma \triangleright e \rightarrow^P \pi_i^P \Omega'; \sigma' \triangleright e'$. We proceed by case analysis on the last rule applied.

- **Case E-AppE1:** We have $e = e_1 e_2$, $e' = e_1' e_2$, and $\Omega; \sigma \triangleright e_1 \rightarrow^P \pi_i^P \Omega; \sigma \triangleright e_1'$. By inversion of typing, $\mathcal{A}, \Gamma \vdash e_1 : T_1 \rightarrow T$ with C_1 and $\mathcal{A}, \Gamma \vdash e_2 : T_1$ with C_2 . By induction hypothesis, $\mathcal{A}, \Gamma \vdash e_1' : T_1 \rightarrow T$ with C_1 . By the typing rule for application, $\mathcal{A}, \Gamma \vdash e' : T$ with C . VM state is unchanged ($\sigma' = \sigma$), so consistency is preserved.
- **Case E-App:** We have $e = (\lambda x : T_1. e_0) v$ and $e' = [v/x]e_0$. By inversion, $\mathcal{A}, \Gamma \vdash \lambda x : T_1. e_0 : T_1 \rightarrow T$ with C_1 and $\mathcal{A}, \Gamma \vdash v : T_1$ with C_2 . By inversion of abstraction typing, $\mathcal{A}, \Gamma, x : T_1 \vdash e_0 : T$ with C_1 . By the substitution lemma, $\mathcal{A}, \Gamma \vdash [v/x]e_0 : T$ with C . VM state is unchanged, so consistency is preserved.
- **Case E-OnV:** We have $e = \text{on } P \ v$ and $e' = v \text{ on } P$. By inversion, $\mathcal{A}, \Gamma \vdash v : T$ with C . By the typing rule for placed values, $\mathcal{A}, \Gamma \vdash v \text{ on } P : T \text{ on } P$ with C .
- **Case E-TakeV:** We have $e = \text{take } v \text{ on } P$ and $e' = v$. By inversion, $\mathcal{A}, \Gamma \vdash v \text{ on } P : T \text{ on } P$ with C . By inversion of placement typing, $\mathcal{A}, \Gamma \vdash v : T$ with C . Therefore, $\mathcal{A}, \Gamma \vdash e' : T$ with C .

- **Case E-COMM_V**: We have $e = \text{comm } v$ on P_l and $e' = v$ on P_r . By inversion, $\mathcal{A}, \Gamma \vdash v$ on $P_l : T$ on P_l *with* C . By well-formedness of $\Omega \models \mathcal{A}$, there exists a valid tie from P_l to P_r . By the typing rule for communication, $\mathcal{A}, \Gamma \vdash v$ on $P_r : T$ on P_r *with* C .
- **Case E-ASLOCAL_V**: We have $e = \text{asLocal } v$ on P_r and $e' = v$. By inversion, $\mathcal{A}, \Gamma \vdash v$ on $P_r : T$ on P_r *with* C . By inversion of placement typing, $\mathcal{A}, \Gamma \vdash v : T$ *with* C . Therefore, $\mathcal{A}, \Gamma \vdash e' : T$ *with* C .
- **Case E-COLL_V**: We have $e = \text{collective } v$ and $e' = v$ on P . By inversion, $\mathcal{A}, \Gamma \vdash v : T$ *with* C . By the typing rule for collective operations, $\mathcal{A}, \Gamma \vdash v$ on $P : T$ on P *with* C .
- **Case E-REPE₂**: We have $e = \text{rep } v \ e_2$, $e' = e_2 \ v$, and $\sigma' = \sigma[\theta \leftarrow v]$ with $\theta = \text{fresh}$. By inversion, $\mathcal{A}, \Gamma \vdash v : T$ *with* C_1 and $\mathcal{A}, \Gamma \vdash e_2 : T \rightarrow T$ *with* C_2 . By the typing rule for application, $\mathcal{A}, \Gamma \vdash e_2 \ v : T$ *with* C . For VM state consistency ([Definition 5.18](#)): $\theta \notin \text{dom}(\sigma)$ by freshness, and $\sigma'(\theta) = v$ with $\mathcal{A}, \Gamma \vdash v : T$ *with* $\emptyset$, satisfying the well-typedness condition for $\text{ty}(c) = T$.
- **Case E-NBR_V**: We have $e = \text{nbr } v$ and $\sigma' = \sigma[\theta \leftarrow v]$ with $\theta = \text{fresh}$. By inversion, $\mathcal{A}, \Gamma \vdash v : T$ *with* C . VM state consistency is preserved: $\sigma'(\theta) = v$ with $\mathcal{A}, \Gamma \vdash v : T$ *with* $\emptyset$, satisfying $\text{ty}(c) = T$ for the `nbr` construct.
- **Case E-BRANCHVTRUE**: We have $e = \text{branch true } e_2 \ e_3$, $e' = e_2$, and $\sigma' = \sigma[\theta \leftarrow \text{true}]$ with $\theta = \text{fresh}$. By inversion, $\mathcal{A}, \Gamma \vdash e_2 : T$ *with* C . VM state consistency: $\sigma'(\theta) = \text{true} \in \{\text{true}, \text{false}\}$, satisfying $\text{ty}(c) = \text{Bool}$. Case **E-BRANCHVFALSE** is symmetric.
- **Case E-HANDLE_V**: We have $e = \text{handle } C \ P \ v$ and $e' = v$. By inversion, $\mathcal{A}, \Gamma \vdash v : T$ *with* C and $C \in \mathcal{C}$. Therefore, $\mathcal{A}, \Gamma \vdash e' : T$ *with* C .
- Other evaluation cases (**E-APPE₂**, **E-ONE**, **E-TAKE_E**, **E-COMME**, etc.) follow similarly by inversion of typing rules and application of induction hypothesis; VM state is unchanged in these congruence rules, so consistency is trivially preserved.

□

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009